

STM32F4 Series self-test library user guide

Introduction

This document applies to the X-CUBE-STL self-test library set for STM32F4 microcontrollers (order code X-CUBE-STL-F4). The X-CUBE-STL-F4 Expansion Package for STM32Cube includes an application-independent software test library. It is released by ST to implement a relevant subset of safety mechanisms required by the safety concepts applicable to STM32F4 microcontrollers that include the Arm® Cortex®-M4.

Table 1. Applicable product

Part number	Order code
X-CUBE-STL	X-CUBE-STL-F4





1 General information

1.1 Purpose and scope

The X-CUBE-STL-F4 Expansion Package runs on STM32F4 Series Arm® Cortex®-M4 based microcontrollers.

arm

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

The version of the application-independent software library (Self-Test Library), available in the X-CUBE-STL-F4 Expansion Package (and associated to this manual), *STL_Lib.a* file, is V2.0.0.

1.2 Reference documents

- [1] UM2490, STM32F4 Series self-test library safety manual
- [2] AN5659, Safety self-test library Arm Cortex-M integration tips
- [3] ES073, X-CUBE-STL software and documentation errata

Note: [3] is kept here for traceability, as it applies to older releases of the X-Cube-STL software.



2 STM32Cube overview

2.1 What is STM32Cube?

STM32Cube is an STMicroelectronics original initiative to improve designer productivity significantly by reducing development effort, time, and cost. STM32Cube covers the whole STM32 portfolio.

STM32Cube includes:

- A set of user-friendly software development tools to cover project development from conception to realization, among which are:
 - **STM32CubeMX**, a graphical software configuration tool that allows the automatic generation of C initialization code using graphical wizards
 - **STM32CubeIDE**, an all-in-one development tool with peripheral configuration, code generation, code compilation, and debug features
 - **STM32CubeProgrammer (STM32CubeProg)**, a programming tool available in graphical and command-line versions
 - **STM32CubeMonitor (STM32CubeMonitor, STM32CubeMonPwr, STM32CubeMonRF, STM32CubeMonUCPD)**, powerful monitoring tools to fine-tune the behavior and performance of STM32 applications in real time
- **STM32Cube MCU and MPU Packages**, comprehensive embedded-software platforms specific to each microcontroller and microprocessor series (such as STM32CubeF4 for the STM32F4 Series), which include:
 - STM32Cube hardware abstraction layer (HAL), ensuring maximized portability across the STM32 portfolio
 - STM32Cube low-layer APIs, ensuring the best performance and footprints with a high degree of user control over hardware
 - A consistent set of middleware components such as ThreadX, FileX / LevelX, NetX Duo, USBX, USB-PD, touch library, network library, mbed-crypto, TFM, and OpenBLRTOS, USB, and graphics
 - All embedded software utilities with full sets of peripheral and applicative examples
- **STM32Cube Expansion Packages**, which contain embedded software components that complement the functionalities of the STM32Cube MCU and MPU Packages with:
 - Middleware extensions and applicative layers
 - Examples running on some specific STMicroelectronics development boards

2.2 How does this software complement STM32Cube?

The software expansion package extends **STM32Cube** by a middleware component to manage specific software-based diagnostics.

The package provides a generic starting point to help a user to build and finalize application specific safety solutions. It consists of:

- **STL**: the self-test library. This provides a binary and some source code to manage the execution of generic safety tests for the microcontroller. The STL is a standalone unit, which runs independently from any STM32 software. It collects the self-tests for generic components of the microcontroller.
- **User application**: This is an STL integration example based on a set of **STM32Cube** drivers extending the STL by an application specific test. This part is delivered as full source code to be adapted or extended by calling of additional application specific modules defined by end user. The example can be used for the library testing including artificial failing support of all the provided modules.



3 STL overview

The STL is an application-independent software test library released by ST to implement a relevant subset of safety mechanisms required by the safety concepts applicable to STM32F4 microcontrollers. The STL is HAL/LL independent, and is dedicated to STM32F4 microcontrollers. The STL is compilation tool chain-agnostic, so it can be compiled by any standard C-compiler.

The STL is an autonomous software, which executes, on application-demand, selected tests to detect hardware issues, and reports the outcomes to the application.

The STL is delivered partly in object code (for the library itself) and partly in source code for the user interfaces definition and user parameters settings.

3.1 Architecture overview

The STL tests the Arm® Cortex® M4 CPU core, the flash memory and the RAM.

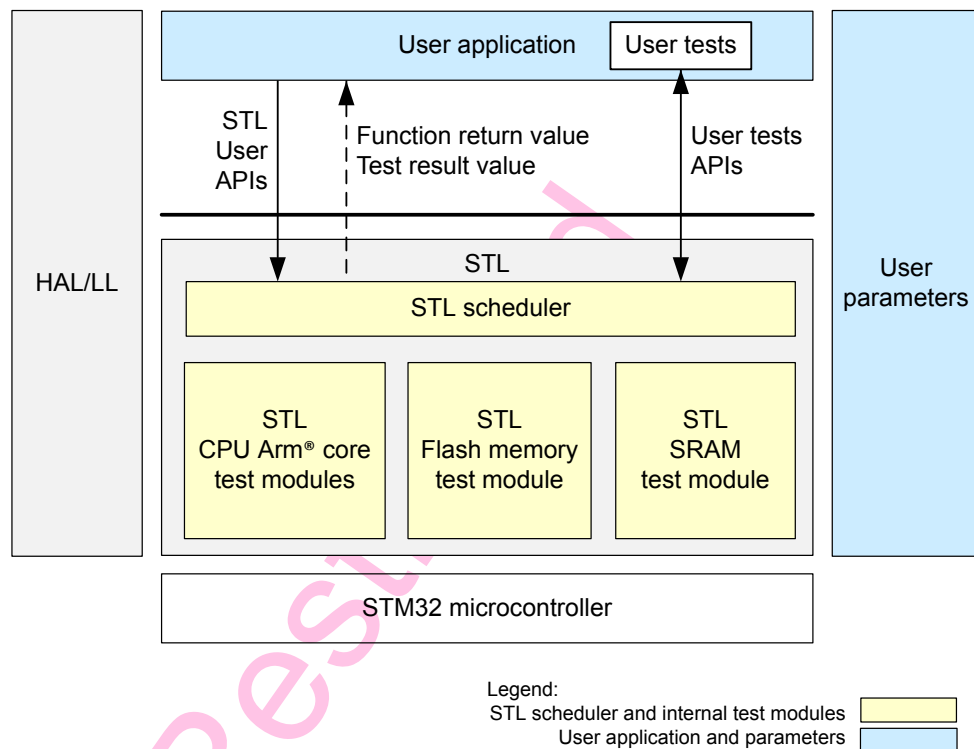
The system architecture of the STL, as shown in [Figure 1. STL architecture](#), is composed of:

- User application (in light blue)
- User parameters (in light blue)
- STL scheduler (in light yellow): directly accessible by the User application via User APIs (not going through HAL/LL)
- STL internal test modules (in light yellow): called by the STL scheduler (not visible from the User application).

The STL status information returned to the User application at API level (see [Table 2. STL return information](#)) are:

- The function return values collect the results of internal defensive programming checks (STL_OK, STL_KO)
- The test module result values store the test result information. This partially corresponds to internal status of the module. (STL_PASSED, STL_PARTIAL_PASSED, STL_FAILED, STL_NOT_TESTED, STL_ERROR).

Figure 1. STL architecture



The STL also allows the developer to:

- Schedule some user tests, that is, tests defined by users and executed through the scheduler. This feature is available through the user configuration.
- Use the artificial-failing feature. The developer can check the application behavior by forcing the STL to return a requested test result value. This feature is available through the specific user API.



3.2 Supported products

The STL runs on the following STM32 microcontrollers:

- STM32F405xx
- STM32F415xx
- STM32F407xx
- STM32F417xx
- STM32F427xx
- STM32F437xx
- STM32F429xx
- STM32F439xx
- STM32F401xC
- STM32F401xE
- STM32F410Tx
- STM32F410Cx
- STM32F410Rx
- STM32F411xE
- STM32F446xx
- STM32F469xx
- STM32F479xx
- STM32F412Cx
- STM32F412Zx
- STM32F412Vx
- STM32F412Rx
- STM32F413xx
- STM32F423xx



4 STL description

This section collects information about the functionality and performance of the STL. It also details the constraints and mandatory actions to be respected by the end user.

4.1 STL functional description

4.1.1 Scheduler principle

The scheduler is the API module needed by the user application to execute the STL.

The main scheduler:

- needs to be initialized before being used
- manages:
 - initialization and de-initialization of the applied test modules
 - configuration of the applied test modules
 - reset of the applied test modules
- controls the execution of:
 - a set of tests configured by the application, for what is called 'multiple testing'
 - a unitary test, for what is called 'single testing'
- manages 'artificial failing' (for user debug and integration tests)
- ensures integrity of critical internal data structures via their specific checksums.

The scheduler controls the execution of the following tests, either in multiple or single testing modes:

- CPU tests: no specific initialization or configuration procedures of the CPU test module are required before CPU test execution (see [Section 7.2 User APIs](#) and [Figure 12. State machine diagram - CPU test APIs](#)). For multiple-test mode, the end user must enable the CPU test modules in the corresponding structure (see [Multiple-test mode](#)).
- flash memory tests over configured subsets (flash memory test module initialization and configuration procedures are mandatory before flash memory test execution, see [Section 7.2 User APIs](#) and [Figure 13. State machine diagram - Flash memory test APIs](#)).
- RAM tests over configured RAM subsets (RAM test module initialization and RAM configuration procedures are mandatory before RAM test execution. See [Section 7.2 User APIs](#) and [Figure 14. State machine diagram - RAM test APIs](#)).
- Specific tests added by the end user, a feature only available in multiple testing. See [Section 4.1.5 User tests](#).

The STL, via the scheduler API, is called by the user in polling mode. The STL can be called in an interrupt context, but re-entrance is forbidden. In such cases, the STL behavior cannot be guaranteed.

The user application has to manage all the returned information from the STL. This is provided via a specific predefined data structure that collects status information. See details in [Table 2](#).

Table 2. STL return information

STL information	Value	Description
Function return value. ⁽¹⁾	STL_OK	Scheduler function successfully executed.
	STL_KO	Scheduler defensive-programming error (in this case the test result is not relevant).
Test-module result value. ⁽²⁾	STL_PASSED	Test passed
	STL_PARTIAL_PASSED	Used only for RAM and flash memory testing, when test is passed, but end of RAM/flash memory configuration not yet reached
	STL_FAILED	Hardware error detection by test module
	STL_NOT_TESTED	Test not executed
	STL_ERROR	Test module defensive-programming error

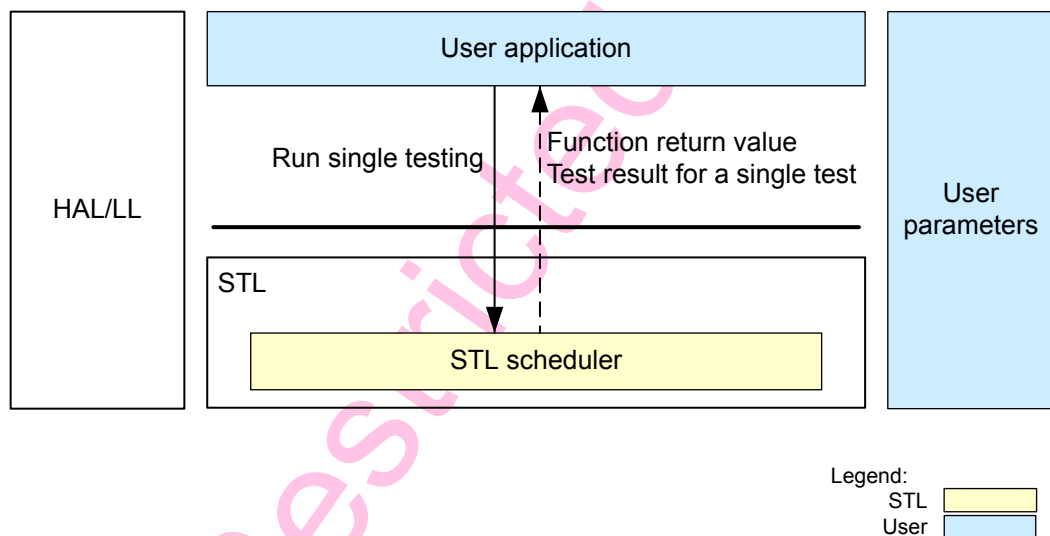
1. See [STL_Status_t](#) definition in [Section 7.1 User structures](#)

2. See Section 7.1 User structures: *STL_TmStatus_t* for single test, or *STL_TmListStatus_t* for multiple test.

Single-test mode

The User application applies the call scheme shown in Figure 2. By doing this repeatedly, the user programs a sequence of single-API function calls, and so handles the order of execution of the test modules.

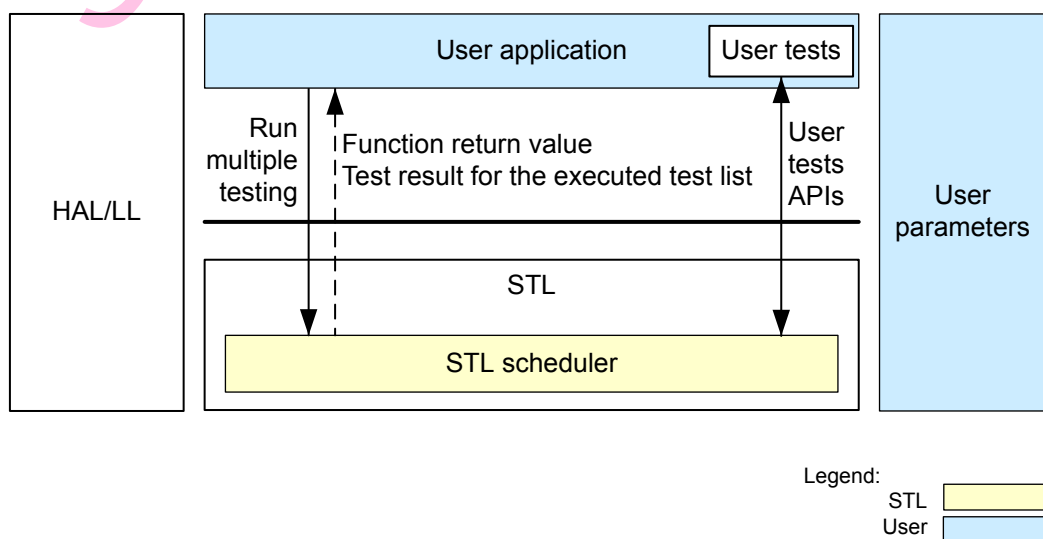
Figure 2. Single test mode architecture



Multiple-test mode

The User application (Figure 3) uses multiple testing API functions (see Section 7.2.5 Multiple testing APIs) to run tests sequentially from a test list, which is configurable by the application. A specific User structure is used to enable the list of TMs to execute during multiple testing (see the definition of *STL_TmListEnable_t* in Section 7.1 User structures).

Figure 3. Multiple-test mode architecture



Scheduler and interrupts

The scheduler is interruptible at any time. Some test modules can temporarily mask interrupts. For more details, refer to Section 4.2.5 STL interrupt masking time, and to Section 4.3.8 Interrupt management.



4.1.2 CPU Arm® core tests

The STL includes the following CPU test modules, listed below, together with a generic description of the test capability:

- TM1: implements the test of general purpose register
- TM1L: implements a simplified version for the test of general purpose register
- TM2: implements the test of arithmetic functions
- TM3: implements the test of multiplier functions
- TM4: implements the test of control flow instructions
- TM5: implements the test of load/store unit(s)
- TM6: implements the test of shift and rotate functions
- TM7: implements the test of both stack pointers, MSP and PSP
- TM7L: implements the test of current stack pointer
- TM8: implements the test of pipeline functions
- TM9: implements the test of the Arm Cortex M4 DSP functions.
- TM10: implements the test of the FPU register bank. This TM must not be used by the application unless the FPU is activated.
- TM11: implements the test of the FPU functions. This TM must not be used by the application unless the FPU is activated.

The tests listed above can be run either in single or multiple test modes.

Caution: The STL CPU tests are partitioned in separated test modules. This allows easy selection of the CPU tests to execute, either one-by-one by using single test mode, or as a sequence of several tests, enabled in a test list, by using multiple test mode. This is not intended to allow partial execution of the overall available CPU TMs, but rather as a support feature to allow better CPU test scheduling in end-user applications (timing constraints for example). To define the list of TMs to be executed, refer to the safety manual [1] (mainly the “Configuration” and “Assumptions” sections), where clear guidance on the TM execution policy is provided..

TM7 and interrupts

The CPU TMs are interruptible at any time.

The CPU TM7 is masking interrupt during the smallest data granularity time (see [Section 4.2.5 STL interrupt masking time](#)).

Refer to [Section 4.3.8 Interrupt management](#) for detailed information on CPU TM7 interrupt management



4.1.3 Flash memory tests

Principles

The Flash test concerns the embedded flash memory of the STM32 MCU.

There are three STL flash memory area concepts:

- Block: a contiguous area of 4 bytes (FLASH_BLOCK_SIZE). The value is hard coded by the STL.
- Section: a contiguous area of 1024 bytes (FLASH_SECTION_SIZE). The value is hard coded by the STL. This has no link with Flash memory physical sector. The Flash memory is partitioned in sections. The first section starts at the first address of the Flash memory, and the next sections are contiguous.
- Binary (named 'user program' in [Figure 5. Flash memory test: CRC use cases versus program areas](#)): a contiguous area of code provided by the compiler. It starts at the beginning of a section. It usually ends with an incomplete section when the binary area size is not a multiple of the section size. In all cases, the binary must be 32-bit aligned (see [ST CRC tool information](#)).
- Subset: a contiguous area of sections with available expected CRC values. The User application can define one subset or several subsets. A subset has to be defined within a binary area. Its start address has to be aligned with a section begin and it can only include sections with a corresponding expected pre-calculated CRC value. When the last subset section is the last part of the binary, the section may be incomplete, and the user application has to align the end of the subset with the end address of the binary area. If a set of complete sections is tested exclusively, the subset end address has to be aligned with the end of the last tested section.
- Subset size = $K * \text{FLASH_SECTION_SIZE} + L * \text{FLASH_BLOCK_SIZE}$, where:
 - K is an integer ≥ 0
 - L is an integer with $0 \leq L < 256$. (256 corresponds to $(\text{FLASH_SECTION_SIZE} / \text{FLASH_BLOCK_SIZE})$. Note that L can exist only when the last section of a binary is incomplete (that is, it is smaller than FLASH_SECTION_SIZE).

The subset is defined by the User application. It is possible to define several subsets.

The STL implements a test of the Flash memory with the following principles:

- tests are performed on sections of the subset(s) defined by the User application
- tests are performed either in a row (one shot) or partially in a single atomic step for a number of sections defined by the User application
- test results are based on a CRC comparison between the computed CRC value (calculated during test execution) and an expected CRC value (calculated before software binary flashing)

The mandatory steps (for the User application) to perform Flash memory tests are:

- test initialization
- configuration of the subset(s)
- execution of the test

Once all subsets are tested, the user needs to reset the Flash memory test module to perform the test again.

In the case of an STL_ERROR / STL_FAILED test result, the test module is stuck at the failed memory subset. The user needs to deinitialize and then reinitialize the Flash memory test module (or reset it) to perform the test again.

Expected CRC pre-calculation

The Flash memory test is based on built-in HW CRC unit or SW CRC, configurable by flag. The default configuration is with HW CRC. To use SW CRC, the flag STL_SW_CRC (see [Section 5.5.2 Steps to build an application from scratch](#)) must be enabled. The CRC is a 32-bit CRC compliant with IEEE 802.3.

Part of the Flash memory is reserved for the CRC dedicated area, the size of which depends on the Flash memory size. This area has a field format where each Flash memory section has sufficient reserved space to store a 32-bit CRC pattern. The user must ensure that valid CRC patterns are calculated and stored in the fields for all the sections to be tested, as shown in [Figure 4. Flash memory test: CRC principle](#).

One expected CRC value is pre-calculated for each contiguous section of a binary, from binary start to binary end. This means that the number of testable sections depends on the binary size. Commonly, the binary area is not aligned with section size, in which case the CRC check value of the last incomplete section is pre-calculated and tested exclusively over the section part that overlays the binary area.

Preconditions:

- The user program areas have to start at the beginning of a section
- The boundaries of the user program areas must be 32-bit aligned.
- Depending on total Flash memory size and on user program size, last program data and first CRC data may be both stored in the same Flash memory section (without any overlap), in that case, CRC must be computed on the user program data only, see Example 3 in [Figure 5. Flash memory test: CRC use cases versus program areas](#).

ST CRC tool information

ST provides such a CRC precalculation tool, available as a single feature inside the STM32CubeProgrammer (see [Section 6.2.2 CRC tool set-up](#)), which automatically fills the binary with padding bits (0x00 pattern) for a 32-bit alignment.

Figure 4. Flash memory test: CRC principle

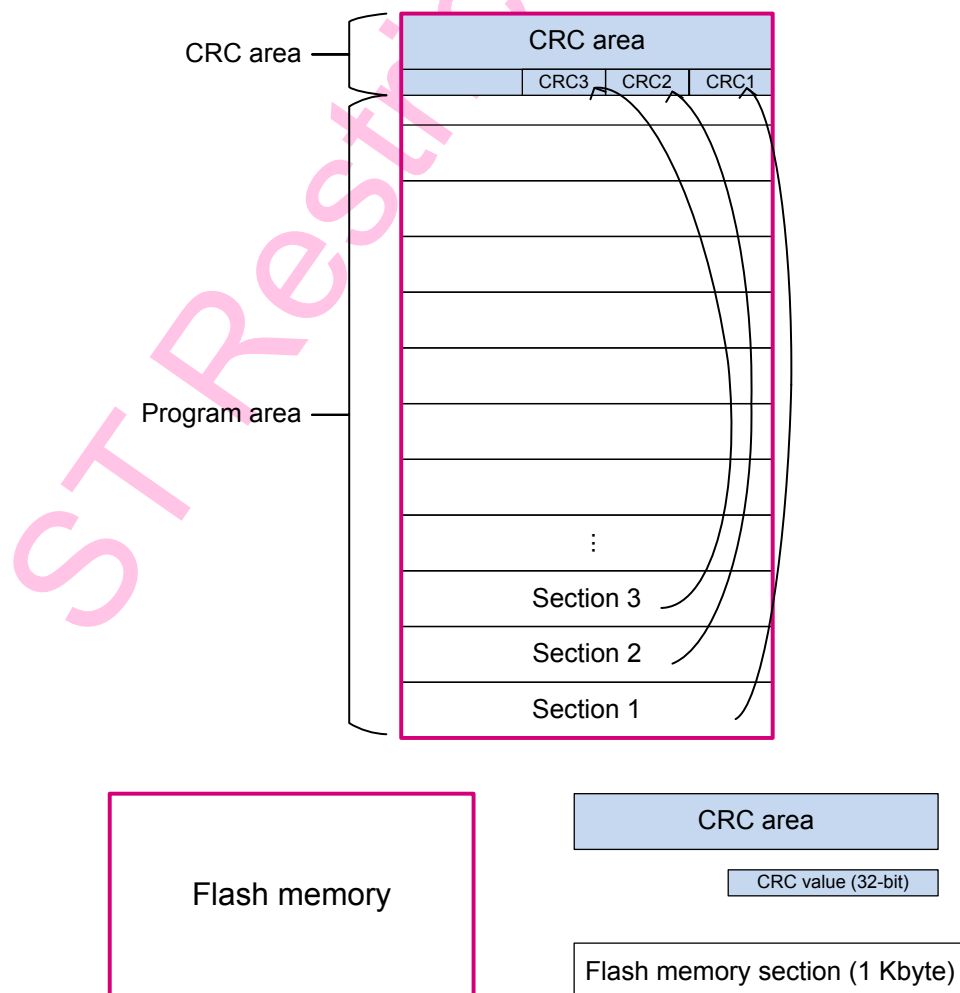
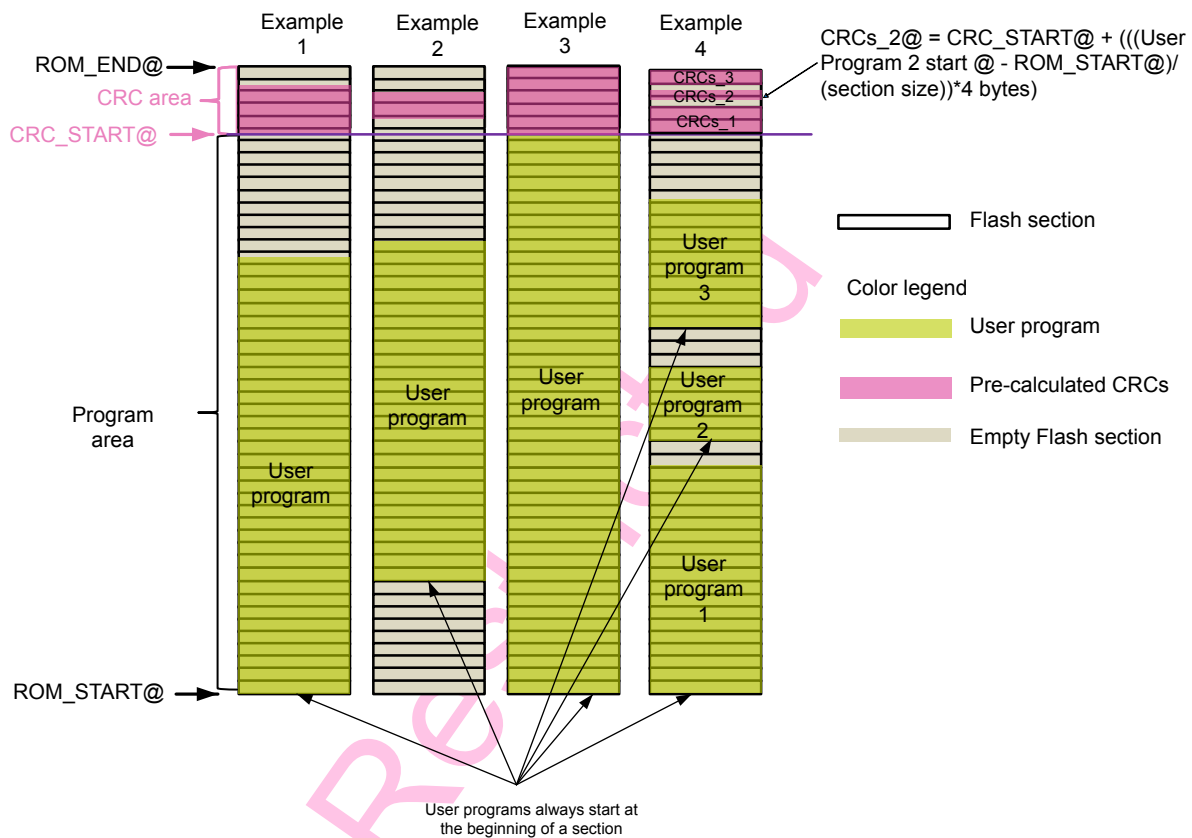


Figure 5. Flash memory test: CRC use cases versus program areas



Use case descriptions:

- Example 1: the user program starts at ROM_START address, so CRCs are stored from CRC_START address.
- Example 2: the user program starts at the beginning of a section, but not at ROM_START. Stored CRCs start at the right address of the CRC area.
- Example 3: the user program uses the full program area, so the last program data and the first CRC data are both stored in the same Flash memory section (without any overlap).
- Example 4: the user program is defined in three separated areas. This requires three separated areas for the CRC data.

CRC start address computation:

- Real calculation:
- $$\text{CRC_START address} = (\text{uint32_t})(\text{ROM_END} - 4 * (\text{ROM_END} + 1 - \text{ROM_START}) / (\text{FLASH_SECTION_SIZE} + 1); \text{ with FLASH_SECTION_SIZE} = 1024$$
- Textual translation:
- $$\text{CRC_START} = \text{ROM_END} - (\text{CRC size in bytes}) * (\text{number of Flash memory sections}) + 1$$

Flash memory test and interrupts

Flash memory TM is interruptible at any time.



4.1.4 RAM tests

Principles

The RAM test concerns the embedded SRAM memories of the STM32 MCU.

There are three RAM area concepts:

- Block: a contiguous area of 16 bytes (RAM_BLOCK_SIZE). This value is hard coded by the STL (there is no link with the memory physical sectors).
- Section: a contiguous area of 128 bytes (RAM_SECTION_SIZE). This value is hard coded by the STL.
- Subset: a contiguous area, with the size being a multiple of two blocks and with a 32-bit aligned start address. Then a subset size is not necessarily a multiple of the section size, because the last part of a subset can be less than one section.
- Subset size = $N * \text{RAM_SECTION_SIZE} + 2 * M * \text{RAM_BLOCK_SIZE}$, where:
 - N is an integer ≥ 0
 - M is an integer with $0 \leq M < 4$, where 4 corresponds to $\text{RAM_SECTION_SIZE} / (2 * \text{RAM_BLOCK_SIZE})$. Note that the case $M > 0$ means that the last part of a subset is not aligned with section size.

The subset is defined by the User application. It is possible to define several subsets.

The STL implements a RAM memory test with the following principles:

- RAM tests are performed on RAM blocks defined by the User application
- RAM tests are performed either in a row (one shot), or partially in a single atomic step for a number of sections defined by the User application
- The test implementation is based on March C- algorithm

The mandatory steps (for the User application) to perform RAM tests are:

- initialization of RAM test
- configuration of the RAM subset(s)
- execution of the RAM test

Once all subsets are tested, the application must reset the RAM test module to perform the test again.

In the case of an STL_ERROR / STL_FAILED test result, the test module is stuck at the failed memory subset. It is therefore advisable to deinitialize then initialize the RAM test module (or to reset it) before performing the test again.

RAM test and interrupts

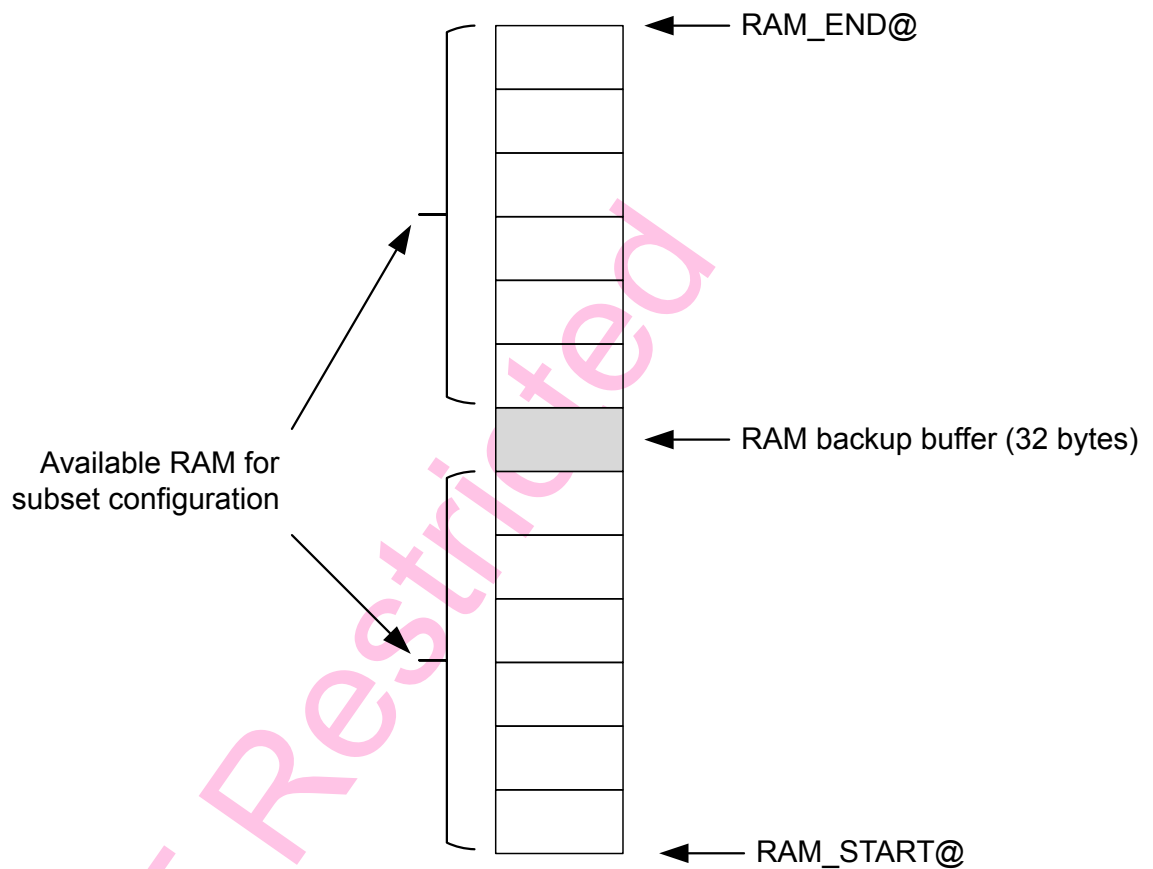
The RAM TM is interruptible at any time except execution of the smallest data granularity (block - see [Section 4.2.5 STL interrupt masking time](#)) when the STM32 interrupts and Cortex® exceptions with configurable priority are temporarily masked by default.

Refer to [Section 4.3.8 Interrupt management](#) for detailed information on interrupt and RAM March-C tests

March C- test principle

The RAM test is based on a March C- algorithm where memory is overwritten by specific patterns and then read back at specific orders. To restore the initial memory content, a backup process is enabled by default. User must allocate specific backup area that is also tested by the March C- test. The backup process can be disabled optionally. Refer to [Section 4.3.11 RAM backup buffer](#) for detailed information on the RAM backup buffer control and allocation.

Figure 6. RAM test: usage



4.1.5

User tests

The user tests have to respect the APIs defined in [Section 7.2.5 Multiple testing APIs](#).

The STL implements a feature, available by configuration, that allows the User application to add tests. This feature is only available for multiple testing. The STL only calls registered user functions with registered user parameters. This means that the user test results are available for the application in the same way as STL internal tests results.



4.2 STL performance data

The following data are obtained with the following test set-up:

- STL library compilation details, detailed in [Section 5.5 Application: compilation process](#).
- Projects for performance tests are compiled with IAR™ Embedded Workbench for Arm® (EWARM) toolchain v9.20.1
- Compiled SW configuration with
 - AHB clock set to 84 MHz
 - Flash latency set to 2 wait states
 - Flash prefetch enabled
 - Nucleo STM32F429ZI, board MB1137 Rev B

4.2.1 STL execution timing summary

Table 3 contains a summary of the STL execution timings. The measurements for each API are detailed in [Section 8 STL: execution timing details](#).

Table 3. STL execution timings, clock at 84 MHz

Tested module	Conditions		Result
CPU	TM1L, TM2, TM3, TM4, TM5, TM6, TM7L, TM8, TM9, TM10, TM11		495 µs
Flash memory	Default configuration (STL_SW_CRC not enabled):	1 Kbyte tested	47 µs
		17 Kbytes tested	588 µs
	STL_SW_CRC enabled:	1 Kbyte tested	199 µs
		17 Kbytes tested	3187 µs
RAM	Default configuration (both STL_DISABLE_RAM_BCKUP_BUF and STL_ENABLE_IT not enabled):	128 bytes tested	83 µs
		256 Kbytes tested	137489 µs

4.2.2 STL code and data size

The STL code and data sizes are detailed in [Table 4. STL code size and data size \(in Bytes\)](#).

Table 4. STL code size and data size (in Bytes)

Configuration	Module	Flash memory code	Flash memory RO-data	R/W data
STL_SW_CRC not enabled, and STL_DISABLE_RAM_BCKUP_BUF not enabled	stl_user_param_template.o	-	6	44
	stl_util.o	246 ¹	-	8
	stl_lib.a	31334	4162	376
STL_SW_CRC enabled, and STL_DISABLE_RAM_BCKUP_BUF not enabled	stl_user_param_template.o	-	6	44
	stl_util.o	154 ¹	-	4
	stl_lib.a	31334	4163	376

1. When SW CRC is used, there are less code/functions compiled (no CRC_Init function with SW CRC), consequently object code size is reduced compared with HW CRC.

4.2.3 STL stack usage

The minimum stack available space required by the STL is 530 bytes.

4.2.4 STL heap usage

The STL never uses dynamic allocation, therefore the heap size is independent from the STL.



4.2.5

STL interrupt masking time

STM32 interrupts and Cortex exceptions with configurable priority are masked multiple times by the STL during CPU TM8 and RAM tests. The maximum interrupt time (see [Table 5](#)) is obtained for a RAM test, when backup buffer is enabled.

Table 5. STL maximum interrupt masking information

Tested module	Duration (max)	Comments
RAM	7 μ s	Each execution of the STL_SCH_RunRamTM function performs series of interrupt masking during partial steps of the test at the following time durations: • 7 μs for the RAM backup buffer • 6 μs for the 1st RAM block to be tested • 7 μs for each middle RAM block to be tested* • 6 μs for the last part RAM block to be tested * Number of RAM blocks (multiple of 2 RAM_BLOCK_SIZE is required) involved at each execution depends on content of user structures (size of defined subset(s) versus atomic step – see Section 4.1.4 RAM tests).
CPU TM7	5 μ s	Masked twice with 5 μ s duration each time.



4.3 STL user constraints

The end user needs to consider interference between the application and the STL. The consequences of ignoring this are possible false STL error reporting, and/or application software execution perturbation.

Accordingly, to prevent any interference the application software and the STL integration must comply with each constraint listed in this section.

4.3.1 Function call convention

The STL complies to function call convention in particular on register save and restore. The application must also respect this calling convention to ensure that the STL does not collect false error reporting.

4.3.2 Privileged-level

The CPU TM7 and the RAM TM must be executed with privileged-level, in order to be able to modify certain core registers (for example the PRIMASK register).

4.3.3 RCC resources

During STL execution, the RCC is configured to clock: the CRC during STL initialization and during TM flash execution

This means that:

- when the STL returns, it restores the user RCC clock setting (enable or disable) for the CRC
- the user application must be careful when configuring the RCC during STL execution by saving/restoring the STL settings.

4.3.4 CRC resources

When the HW CRC is used, the STL relies on the STM32 CRC IP. The CRC resources are used during STL execution in two different cases:

- During execution of STL initialization (function STL_SCH_Init): the HW CRC is used. The use of HW CRC in this phase can't be modified by the application software, so the STL_SW_CRC flag has no impact during execution of the STL_SCH_Init function.
- During execution of Flash TM, the application can choose between HW CRC and SW CRC by means of the STL_SW_CRC flag. By default, HW CRC is used (the STL_SW_CRC flag is disabled).

The use of HW CRC means that:

- Before calling the STL, the user application has to save its own HW CRC configuration, and to restore it after STL execution.
- During STL execution, the HW CRC is configured and used for STL needs (the user application must take care to save/restore the STL settings when using the CRC during STL execution).

4.3.5 Bit Q of APSR

CPU TM2 and TM9 execution can cause bit Q of the APSR (sticky bit) to be set. The user application must take this into account when using this bit.

4.3.6 GE bits of APSR

During CPU TM9 execution, the GE bits of APSR can be set. They are always set to 0 at the end of the CPU TM9.



4.3.7 FPU resources (Cortex M4 FPU block)

During STL CPU TM11 execution, the FPSCR is configured.

This means that:

- when the STL returns, it restores the user FPSCR content
- the user application must take care during interrupt service routines, to save/restore the STL settings for the FPU when configuring the FPSCR during STL execution.

4.3.8 Interrupt management

Escalation mechanism - Arm Cortex behavior reminder

When the STL disables STM32 interrupts and Cortex exceptions with configurable priority, remember that an Arm Cortex escalation to HardFault might occur. In this case, the HardFault handler is called instead of the fault handler.

Interrupt and CPU TM7

By default, the STM32 interrupts, and Cortex exceptions with configurable priority, are temporarily masked during CPU TM7 execution during smallest data granularity (a few instruction blocks), except when the User application activates the dedicated compilation switch, STL_ENABLE_IT (see [Section 5.5.2 Steps to build an application from scratch](#)). If the STL_ENABLE_IT flag is activated, the correct STL CPU TM7 behavior is not warranted. The end user is responsible for managing interferences between the STL and its application software that could possibly lead to false STL error reports, or non-detection of hardware failure.

The CPU test modules unrelated to TM7 are neither affected by masking of STM32 interrupts, nor of Cortex exceptions with configurable priority.

Interrupt and RAM March C- tests

By default, the STM32 interrupts and Cortex exceptions with configurable priority are masked during the RAM March C- tests, except if the user application activates STL_ENABLE_IT (see [Section 5.5.2 Steps to build an application from scratch](#)).

If the STL_ENABLE_IT flag is activated:

- The correct STL RAM test behavior is not warranted, as the STL-tested RAM content may be overwritten by the application during its interrupt treatment (this could lead the STL to generate false RAM test error reporting)
- The User application software behavior can be compromised, because wrong data may be read or used from RAM locations being modified by the STL March C- test.

4.3.8.1 How the STL masks the interrupts

To mask the interrupts, the STL sets the PRIMASK register bit to 1. Setting this bit to 1 boosts the current execution priority to 0, preventing the activation of all exceptions with configurable priority. Thus when the current execution priority is boosted to a particular value, all exceptions with a lower or equal priority are masked.



4.3.9 DMA

The application must manage DMA to avoid unwanted accesses to RAM bank during STL March C- test. In this case:

- DMA writes can disturb the STL test causing false error reporting
- DMA reads can return wrong data, due to STL overwrites to DMA dedicated RAM sections.

4.3.10 Supported memories

Only STM32F4 internal memories are supported by the memory tests (Flash TM and RAM TM).

The STL library must be executed from STM32F4 internal memories only.

4.3.11 RAM backup buffer

The backup process is enabled by default during RAM tests to preserve the RAM content. User must reserve a specific area for the RAM backup buffer at compilation time. This area must be allocated outside of the RAM subset configuration. There is only one RAM backup buffer defined for the test. The RAM backup buffer area is also tested by the March C- test each time the RAM test function is called (before any subset defined by the user is tested).

The backup process can be disabled optionally, either permanently by activating the compilation switch `STL_DISABLE_RAM_BCKUP_BUF` (see [Section 5.5.2 Steps to build an application from scratch](#)), or suppressed temporarily by a specific control sequence detailed in [2]. In this case, it is the end-user's responsibility to guarantee that the application software does not need to use data destroyed by the March C- test. This option can be used to speed up testing of those RAM areas where the user doesn't need to preserve the memory content.

4.3.12 Memory mapping

Due to the RAM test module and March C method design, the user must ensure, via proper adaptation of the associated linker file, that the “read only” data of the STL is located in the Flash memory.

The examples below are for EWARM and SW4STM32 (system workbench for STM32):

EWARM .icf file adaptation example

```
place in ROM_region { readonly };
```

SW4STM32 .ld file adaptation example

```
.rodata :
{
.....
} >FLASH
```

Note: Usually the default configuration locates “read only” data in Flash memory.

4.3.13 Processor mode

The STL CPU TM7 must be executed in thread mode in order to set the active stack pointer to the process stack pointer.

If the STL is not executed in thread mode, CPU TM7 returns `STL_ERROR`.



4.3.14

Integration tips when switching to STL_Lib.a v2.0, and associated source files

This chapter is not intended for new projects. However, it is useful for project integration when switching from a previous *STL_Lib.a* file version, to version *STL_Lib.a* v2.0 and above:

- STL library: The library is compiled with EWARMFS 9.20.3 that defines `wchar_t` type size to 32 bits (while it was 16 bits with previous compiler and STL_Lib versions). Therefore, depending on the project options, it might be necessary to add a flag to define the `wchar_t` type size as 32 bits.
- The CRC (HW or SW): This is no longer used internally by the scheduler. (An exception is during the `STL_SCH_Init` function, where HW CRC is still accessed.). The previous related constraint for application software is therefore relaxed.
- TM7 and TM7L availability: The user must define the TM7 version to apply (see [1] for considerations related to the TM selection), and use the corresponding APIs. The user data structures contain a new item with TM7L. This needs to be considered for multiple testing and artificial failing, because the CPU TM enum (`STL_CpuTmxIndex_t`) size (`STL_CPU_TM_MAX`) is increased. See [Section 7.1 User structures](#).
- The new CPU TM7 version: This adds a constraint on the application software, by masking interrupts. This is not the case for TM7L (previously named TM7).
- CPU TM8: Reworked to remove interrupt disable intervals, and hence the related constraint on the application software.
- CPU TM9: Correction for a limitation related to GE bits from APSR affecting previous versions. Therefore, the related workaround, described in [3], is no longer needed.



4.4 End user integration tests

This section describes the mandatory tests to be executed by the end user during the verification phase, to guarantee that the STL is correctly integrated in the application software.

4.4.1 Test 1: correct STL execution

The end user must use the expected function return value and the expected test module result value (see [Section 7.2 User APIs](#)), to check that each planned diagnostic function (for example run CPU tests, configure RAM tests or reset flash memory tests), has been correctly executed.

4.4.2 Test 2: correct STL error-message processing

The end user must check that any error information produced by the STL function-return and test-module-result values (that is, values different to the expected value, see [Section 7.2 User APIs](#)), is correctly interpreted as unexpected behavior, and correctly handled in its application software. During the verification, the artificial-failing feature must be used to emulate the generation of incorrect test-module result values related to associated individual software diagnostics (CPU tests, RAM test, flash memory test), for each of the individual functions used.

This process cannot be considered as an exhaustive simulation of actual CPU failures on real devices. It is rather a testing interface for the implemented APIs.

5 Package description

This section details the X-CUBE-STL-F4 Expansion Package content and its correct use.

5.1 General description

X-CUBE-STL-F4 is a software Expansion Package for STM32F4 Series microcontrollers.

It provides a complete solution that helps end customers to build a safety application:

- an application-independent software test library is available:
 - partly as object code: *STL_Lib.a*, the library itself
 - partly as source file: *stl_user_param_template.c* and *stl_util.c*
 - with three header files: *stl_stm32_hw_config.h*, *stl_user_api.h*, and *stl_util.h*
- a User application example, available as source code

X-CUBE-STL-F4 has been ported on the products listed in [Section 3.2 Supported products](#).

The software Expansion Package includes a sample application that the developer can use to start experimenting with the code. It is provided as a zip archive containing both source code and library.

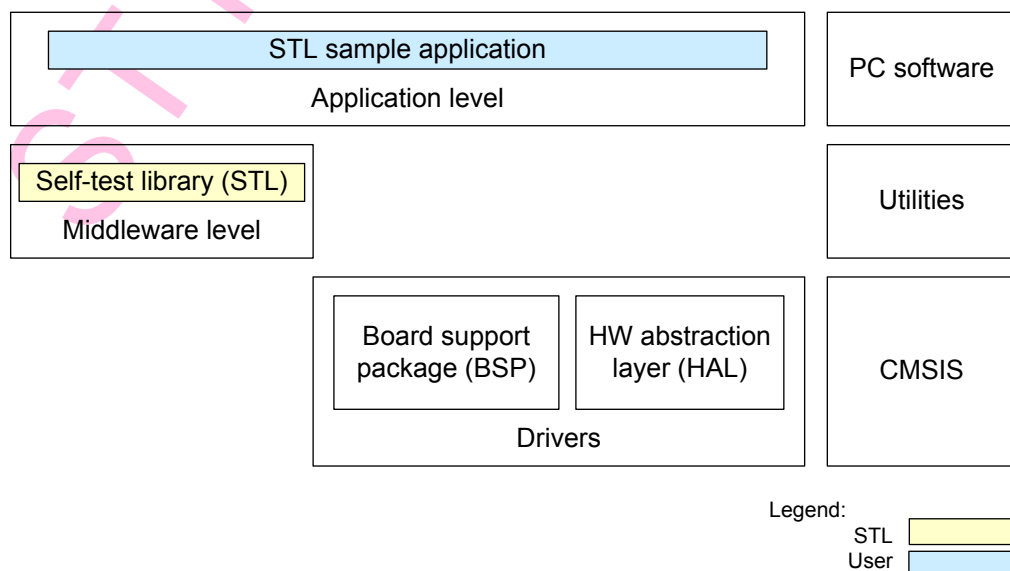
The following integrated development environments are supported:

- IAR Embedded Workbench® for Arm® (EWARM).
- Keil® microcontroller development kit (MDK-ARM)
- System workbench for STM32 (SW4STM32)

5.2 Architecture

The components of the X-CUBE-STL-F4 Expansion Package are illustrated in [Figure 7](#).

Figure 7. Software architecture overview





5.2.1 STM32Cube HAL

The HAL driver layer provides a simple, generic, multi-instance set of APIs (application programming interfaces) to interact with the upper layers (application, libraries and stacks).

It comprises generic and extension APIs. It is directly built around a generic architecture and allows the layers that are built upon, such as the middleware layer, to implement their functionalities without dependencies on the specific hardware configuration of a given microcontroller.

This structure improves the library code re-usability and guarantees an easy portability to other devices.

5.2.2 Board support package (BSP)

The software package needs to support the peripherals on the STM32 boards, apart from the MCU. This software is included in the board support package (BSP). This is a limited set of APIs that provides a programming interface for some specific board components, such as the LED and the user button.

5.2.3 STL

The STL is available at middleware level. It can mainly (for the STL library part) be considered a black box that manages the software-based diagnostic test. The STL is independent of the HAL, BSP, and CMSIS, though the STL integration examples rely on some HAL drivers.

5.2.4 User application

The examples provided show how to:

- use the various APIs with different test modes and features
- integrate a possible sequence of STL test module calls into an application,
- verify the returns of the APIs and structures
- emulate the failure responses artificially.

The examples are provided solely as a quick demonstration of how to call some STL APIs when adopting different IDEs. They must not be used as representative templates for the integration and use of STLs in real safety-application software.

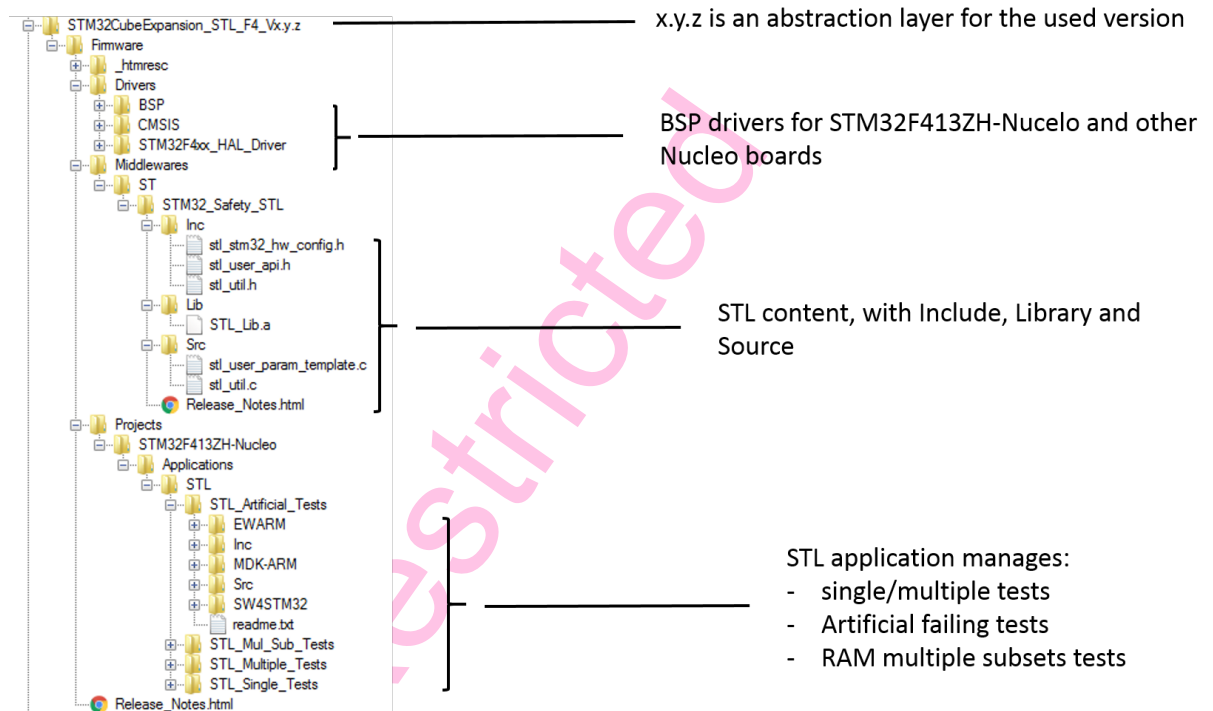
5.2.5 STL integrity

The integrity of the STL content is ensured by hash SHA-256.

5.3 Folder structure

A top-level view of the structure is shown in Figure 1.

Figure 8. Project file structure





5.4 APIs

5.4.1 Compliance

Interface compliance

The library part of the STL, not delivered in source code, has been compiled with IAR Embedded Workbench® for Arm® (EWARM) v9.20.3 (certified version) with `--aeabi` and `--guard_calls` compilation options to fulfill AEABI compliance as described in “AEABI compliance” of EWARM help section.

Safety guidelines compliance

The library part of the STL, not delivered in source code, has been compiled with IAR Embedded Workbench® for Arm® vxxx EWARMFS (certified version) with `--strict`, `--remarks`, `--require_prototypes`, and `--no_unaligned_access` compilation options to fulfill the safety guideline compliance as described in the IAR embedded workbench safety guide (Advices 2.1-1, 2.2-5, 2.4-1a and 5.4-3) and Keil® safety manual (§4.9.2).

Library compliance

The library part of the STL (not delivered in source code) is conformant with C Standard Library ISO C99. It has been compiled with the IAR™ option `. language C dialect = Standard C`.

Arm compiler C toolchain vendor/version independency

The STL user API refers only to the “uint32_t” and “enum” C types:

- “uint32_t” C type is a fixed type size of 32 bits according to C Standard C99
- “enum” C type size, according to C Standard C99, is implementation-defined. It must be capable of representing the values of all the enumeration members. In the STL interface, the enum type values are unsigned integers, smaller than or equal to $(2^{32} - 1)$. The user must ensure that enum type value can hold a 32-bit value.

5.4.2 Dependency

- The STL Library calls the `memset` Standard C library function.
- Furthermore, the IAR EWARM toolchain compiler (used to compile the STL library) may, under some circumstances, call the following Standard C library functions: `memcpy`, `memset` and `memclr`. This behavior is intrinsic to the IAR EWARM toolchain compiler (it is not possible to disable or avoid it).

As a result, when linking the STL Library the user must ensure that these Standard C library functions are defined (use either those provided by the toolchain or the user's own ones).

5.4.3 Details

Detailed technical information about the available APIs can be found in [Section 7.2 User APIs](#), where the functions and parameters are described.



5.5 Application: compilation process

5.5.1 Steps to build a delivered STL example

1. Install the ST CRC tool (see [Section 6.2.2 CRC tool set-up](#)), used for Flash test. If the customer prefers to use their own CRC tool, some steps specific to ST tools are no longer needed.
2. Project choice. Select a project example and open it
3. Project build. Launch the build that:
 - a. compiles a binary
 - b. calculates the CRCs (in the case of error, check the CRC tool path), for details see [Section 5.5.2 Steps to build an application from scratch](#)
 - c. automatically adds the computed CRCs to the compiled binary
4. Loading. Load the compiled binary
5. Execution. Boot the board, the test result is:
 - a. LED ON: test result as expected
 - b. LED toggling: error (except for artificial case)

Note: After the compilation/link of a binary, the CRC tool used in the post-build script:

- Calculates the CRCs (in the case of error, check the CRC tool path), for details see [Section 5.5.2 Steps to build an application from scratch](#).
- Automatically adds the computed CRCs to the generated binary.

5.5.2 Steps to build an application from scratch

1. Install the ST CRC tool (see [Section 6.2.2 CRC tool set-up](#)), used for Flash test. If the customer prefers to use their own CRC tool, some steps specific to ST tools are no longer needed.
2. Create new application project with the necessary directory structure and the required packages. Use the STM32CubeMX tool to make it automatically.
3. Copy paste the content of the ...Middleware\ST\STM32_Safety_STL directory from the delivered STL example into the application project directories structure. Refer to [Section 5.3 Folder structure](#). Then modify the project settings while following the next steps:
 - a. add all the STL source files located at the SRC directory into the project
 - b. assign the INC directory as an additional include one at the project, and
 - c. force the linker to include the library object file located at the LIB directory as an additional library.
4. If needed, add the next optional preprocessor compilation switches to the project settings:
 - a. Option to enable STL_DISABLE_RAM_BCKUP_BUF, if the RAM backup buffer is not used (in this case the RAM data of the tested subsets are destroyed). If not activated, the RAM backup buffer is used by default. In such cases, the "backup_buffer_section" section must be defined in the scatter file.
 - b. Option to enable STL_SW_CRC, if SW CRC is used. If not activated, the HW CRC is used by default.
 - c. Option to enable STL_ENABLE_IT, if the User application wants to enable the STM32 interrupts and Cortex exceptions with configurable priority during the CPU TM7 and the RAM test. If not activated, the interrupts are masked during these tests. Note that if the switch is activated, the end user must evaluate potentially related issues (as described in [TM7 and interrupts](#) and [Section 4.1.4 RAM tests](#)).
 - d. It is mandatory to enable the STM32 device part number, to select the correct HW CRC configuration and the correct Flash memory density.
5. Check Flash memory density configuration. It is mandatory to:
 - a. Set the correct range of the flash memory for the project in the `stl_user_param_template.c` file. Ensure coherency especially with the associated linker scatter file and CRC tool script (see step 6).
 - b. Check the configuration file `stl_user_param_template.c` and potentially to update the `STL_ROM_END_ADDR` to the correct size of the Flash memory (coherency with the associated linker scatter file).

6. Develop the user application by implementing the proper sequence of API calls repeated at periodical cycles, as required by the defined safety task.
It is mandatory to ensure a proper filling of all the associated user structures to control the memory tests and apply a correct check of the STL return information. [Section 7 STL: User APIs and state machines.](#)
7. Apply the CRC tool to build the CRC area content necessary for the CRC calculation. ST CRC calculations are done either by post build (within the IDE), or with a command line:
 - a. Post-build example with IAR IDE: In the project "Option", select "Build Actions", and add in post-build the "one line" command line (as shown in [Section 5.5.2](#))
 - b. Command line example: Use the same command line shown in Figure 10. CRC tool command line.
In the case of an error, check the CRC tool path.
8. Loading. Load the compiled binary.
9. Compile, load, and execute the binary.

Remember: During the verification, the artificial-failing feature (see [Section 7.2.6 Artificial-failing APIs](#)) must be used to emulate the generation of incorrect test-module result values related to associated individual software diagnostics (CPU tests, RAM test, Flash memory test), for each of the individual functions used. See [Section 4.4.2 Test 2: correct STL error-message processing.](#)

Figure 9. IAR post-build actions screenshot

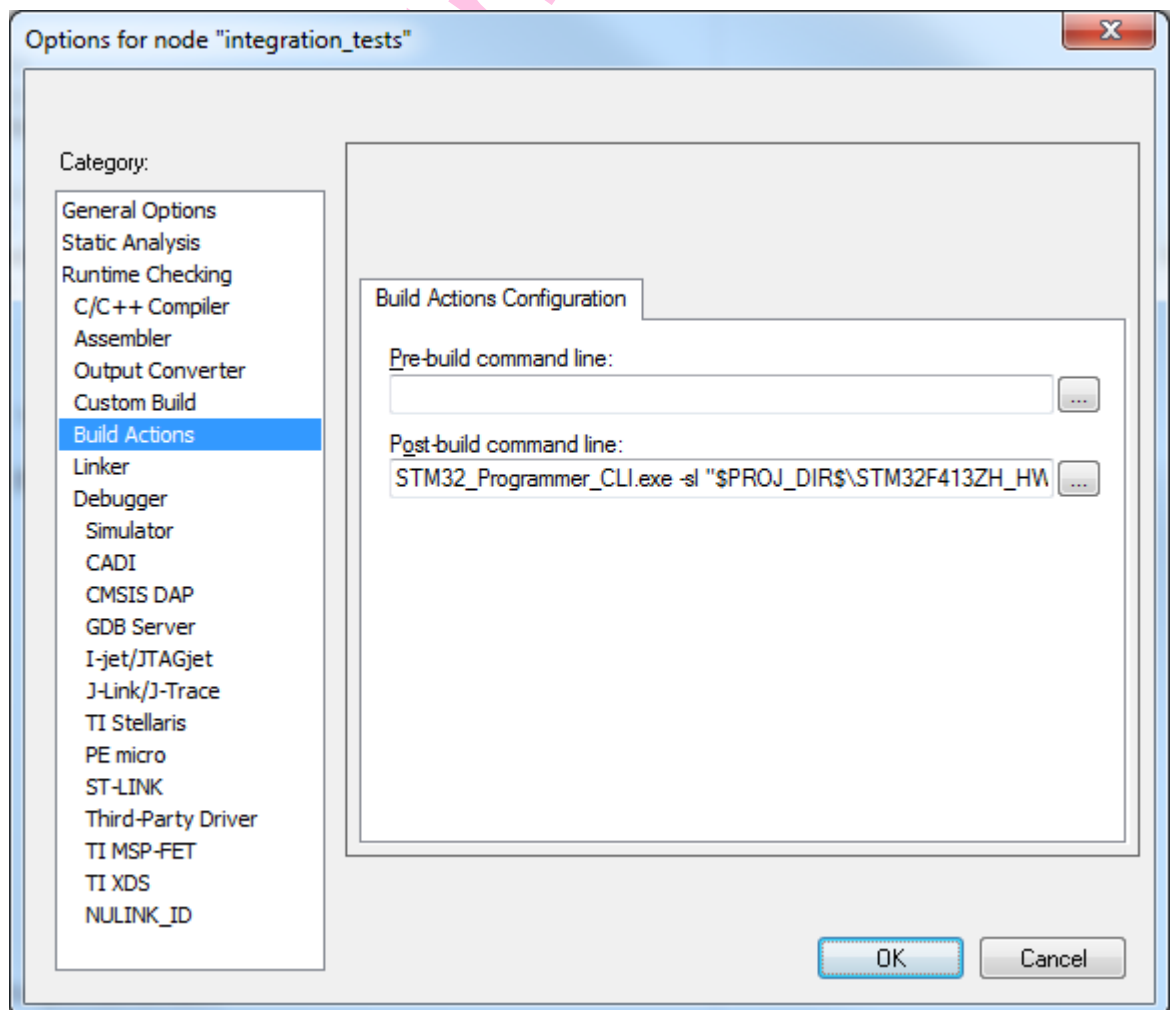
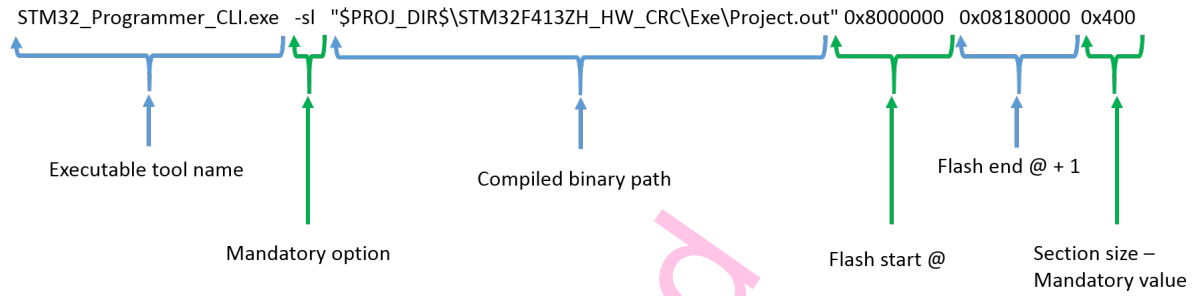




Figure 10. CRC tool command line



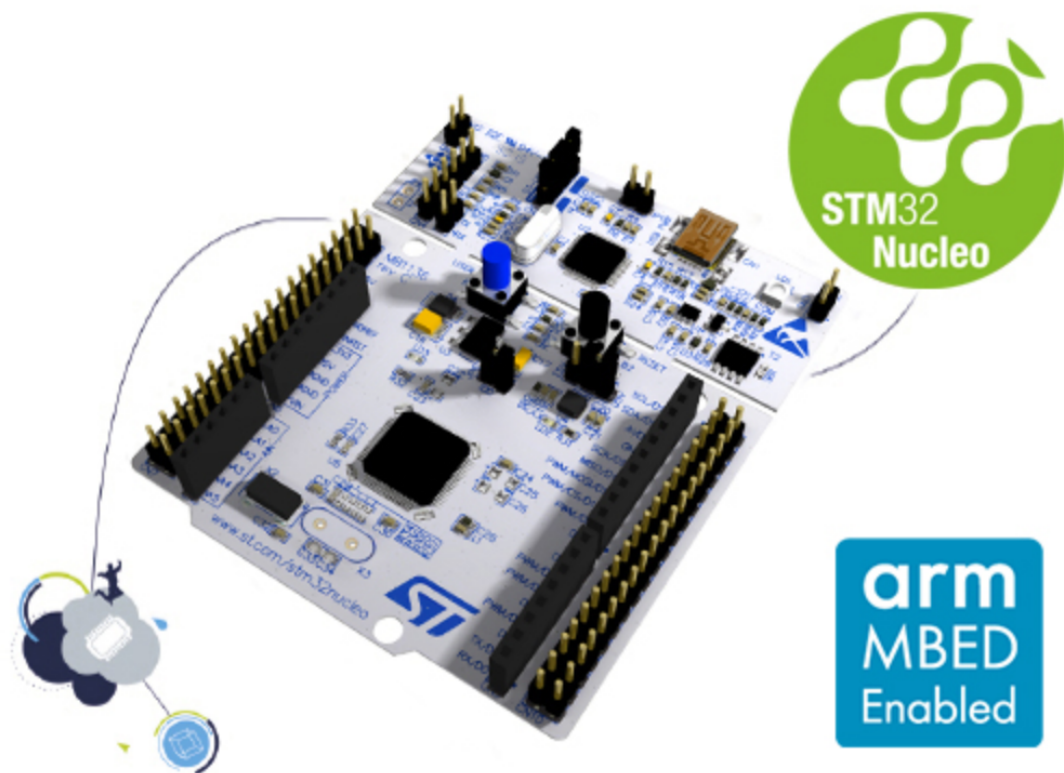
6 Hardware and software environment setup

6.1 Hardware setup

The STM32 Nucleo boards provide an affordable and flexible way for users to try out new ideas and build prototypes with any STM32 microcontroller lines. The Arduino™ connectivity support and ST morpho headers make it easy to expand the functionality of the STM32 Nucleo open development platform with a wide choice of specialized expansion boards. The STM32 Nucleo board does not require any separate probe as it integrates the ST-LINK/V2-1 debugger/programmer. The STM32 Nucleo board comes with the STM32 comprehensive software HAL library together with various packaged software examples.

Details about the STM32 Nucleo boards (see Figure 11) are available from the <http://www.st.com/stm32nucleo> web page.

Figure 11. STM32 Nucleo board example



The following components are needed:

- STM32 NUCLEO-STM32F429ZI development board
- USB type A to Mini-B USB cable to connect the STM32 Nucleo to the PC.



6.2 Software setup

This section lists the minimum requirements for the developer to setup the SDK, run the sample scenario and customize applications.

6.2.1 Development tool-chains and compilers

Select one of the IDEs supported by the STM32Cube software Expansion Package.

Read the system requirements and setup information provided by the selected IDE provider.

Check the projects *Release_Notes.html* file inside the release package, and refer to the "IDE compatibility" chapter, if it exists.

6.2.2 CRC tool set-up

A CRC tool that can be used to generate the CRC checksums needed for X-CUBE-STL flash memory testing is available as a single feature inside the STM32CubeProgrammer. Other CRC tools can be used, provided they fulfill the requirements detailed in [Expected CRC pre-calculation](#).

Tool installation procedure:

- Select STM32CubeProgrammer on the dedicated web page available on www.st.com
- Install the package.

The easiest way is to add the tool path inside the environment variable (computer administration rights are required). If not, remember the path, as it is directly added in the project for compilation, in the post-build option.



7 STL: User APIs and state machines

7.1 User structures

The structures are defined in *stl_user_api.h*. It is forbidden to change the content of this file.

Structures detailed hereafter are a copy of the *stl_user_api.h* content:

```
typedef enum
{
    STL_OK = STL_OK_DEF, /* Scheduler function successfully executed */
    STL_KO = STL_KO_DEF /* Scheduler function unsuccessfully executed
                        (defensive programming error, checksum error). In this case
                        the STL_TmStatus_t values are not relevant */
} STL_Status_t; /* Type for the status return value of the STL function execution */
```

```
typedef enum
{
    STL_PASSED = STL_PASSED_DEF, /* Test passed. For Flash/RAM, test is passed and end of
                                configuration is also reached */
    STL_PARTIAL_PASSED = STL_PARTIAL_PASSED_DEF, /* Used only for RAM and Flash testing.
                                                Test passed, But end of Flash/RAM
                                                configuration not yet reached */
    STL_FAILED = STL_FAILED_DEF, /* Hardware error detection by Test Module */
    STL_NOT_TESTED = STL_NOT_TESTED_DEF, /* Initial value after a SW init, SW config,
                                         SW reset, SW de-init or value when Test Module
                                         not executed */
    STL_ERROR = STL_ERROR_DEF /* Test Module unsuccessfully executed (defensive programming
                              check failed) */
} STL_TmStatus_t; /* Type for the result of a Test Module */
```

```
typedef enum
{
    STL_TEST_ENABLE = STL_TEST_ENABLE_DEF, /* used to add a Test Module in the list to run */
    STL_TEST_DISABLE = STL_TEST_DISABLE_DEF /* used to remove a Test Module from the list
                                             to run*/
} STL_TmEnable_t; /* Type used/checked only for Multiple testing, to add/remove a test
module
in/from the test list */
```

```
typedef enum
{
    STL_CPU_TM1_IDX = 0U, /* CPU Arm Core Test Module 1 index */
    STL_CPU_TM1L_IDX, /* CPU Arm Core Test Module 1L index */
    STL_CPU_TM2_IDX, /* CPU Arm Core Test Module 2 index */
    STL_CPU_TM3_IDX, /* CPU Arm Core Test Module 3 index */
    STL_CPU_TM4_IDX, /* CPU Arm Core Test Module 4 index */
    STL_CPU_TM5_IDX, /* CPU Arm Core Test Module 5 index */
    STL_CPU_TM6_IDX, /* CPU Arm Core Test Module 6 index */
    STL_CPU_TM7_IDX, /* CPU Arm Core Test Module 7 index */
    STL_CPU_TM7L_IDX, /* CPU ARM Core Test Module 7L index */
    STL_CPU_TM8_IDX, /* CPU Arm Core Test Module 8 index */
    STL_CPU_TM9_IDX, /* CPU Arm core test module 9 index */
    STL_CPU_TM10_IDX, /* CPU Arm core test module 10 index */
    STL_CPU_TM11_IDX, /* CPU Arm core test module 11 index */

    STL_CPU_TM_MAX /* Number of CPU Arm Core Test Modules */
} STL_CpuTmIndex_t; /* Type for index of CPU Arm Core Test
Modules */
```



```
typedef struct STL_UserTmConfig_struct /* Structure tag that define a User Test */
{
    uint32_t TmId; /* ID of one User Test */
    STL_TmStatus_t (*pFunc)(void *FuncParam); /* Pointer to user function to be run by the
                                                STL scheduler */
    void *pFuncParam; /* Pointer to user function parameters */
    struct STL_UserTmConfig_struct *pNext; /* Pointer to next User tests configuration - to be
                                                set to NULL for the last user function */
} STL_UserTmConfig_t; /* Type used to define User
Tests test list - only useful during multiple testing - if not used, to be set to NULL */
```

```
typedef struct STL_UserTmStatus_struct /* Structure tag that define a User Test
                                        test status */
{
    uint32_t TmId; /* ID of one User Test */
    STL_TmStatus_t TmStatus; /* Test status of the User Test corresponding to this ID */
    struct STL_UserTmStatus_struct *pNext; /* Pointer to next User Test test status
                                                definition */
} STL_UserTmStatus_t; /* Type used to define User Tests status */
```

```
typedef struct
{
    STL_TmStatus_t aCpuTmStatus[STL_CPU_TM_MAX]; /* array for CPU Arm Core Test
                                                Module test status */
    STL_TmStatus_t FlashTmStatus; /* Flash Test Module status */
    STL_TmStatus_t RamTmStatus; /* RAM Test Module status */
    STL_UserTmStatus_t *pUserTmStatus; /* Pointer to User Tests status */
} STL_TmListStatus_t; /* Type used for Test Modules status */
```

```
typedef struct
{
    STL_TmEnable_t aCpuTmEnable[STL_CPU_TM_MAX]; /* array for CPU Arm Core Test
                                                Module test list */
    STL_TmEnable_t FlashTmEnable; /* Flash Test Module enablement information */
    STL_TmEnable_t RamTmEnable; /* RAM Test Module enablement information */
} STL_TmListEnable_t; /* Type used to define STL internal tests list */
```

```
typedef struct STL_MemSubset_struct
{
    uint32_t StartAddr; /* start address of Flash or RAM memory subset */
    uint32_t EndAddr; /* end address of Flash or RAM memory subset */
    struct STL_MemSubset_struct *pNext; /* pointer to the next Flash or RAM memory subset
                                                - to be set to NULL for the last subset */
} STL_MemSubset_t; /* Type used to define Flash
or RAM subsets to test */
```

```
typedef struct
{
    STL_MemSubset_t *pSubset; /* Pointer to the Flash or RAM subsets to test */
    uint32_t NumSectionsAtomic; /* Number of Flash or RAM sections to be tested
                                during an atomic test */
} STL_MemConfig_t; /* Type used to fully define Flash or RAM test configuration */
```

```
typedef struct
{
    STL_TmStatus_t aCpuTmStatus[STL_CPU_TM_MAX]; /* Array of forced status value
                                                for CPU Test Modules */
    STL_TmStatus_t FlashTmStatus; /* Forced status value for Flash Test Module */
    STL_TmStatus_t RamTmStatus; /* Forced status value for RAM Test Module */
} STL_ArtifFailingConfig_t; /* Type used to force Test Modules status to a specific
value for each STL Test Module */
```




7.2 User APIs

The following APIs are defined in the file *stl_user_api.h*. It is forbidden to change the content of this file.

Caution: For pointers defined by the user application and used as STL API parameters, the user application must set valid pointers, maintain pointer availability, and check the pointer integrity. The STL does not copy the pointer content, and directly accesses memory addresses defined by the application.

This applies during overall STL execution, where, for example, the pointers content for Flash and RAM configuration must be maintained because they are still used by the STL_SCH_run_xxx functions, even if these pointers are not part of the input parameters.

For more details about proper sequences of the API calls see [Section 7.3 State machines](#) and [Section 7.6 Test examples](#).

7.2.1 Common APIs (used whatever the test mode)

7.2.1.1 *STL_Status_t* STL_SCH_Init

Description: initializes the scheduler. It can be used at any time to re-initialize the scheduler (it resets all tests).

Declaration: *STL_Status_t* STL_SCH_Init(void)

Table 6. STL_SCH_Init input information

Allowed state(s)	Parameter(s)
CPU TMx: all Flash TM: all Ram TM: all	-

Table 7. STL_SCH_Init output information

STL_Status_t return value		Returned state
Value	Comment(s)	
STL_OK	Function successfully executed	CPU TMx: CPU_TMx_CONFIGURED Flash TM: FLASH_IDLE Ram TM: RAM_IDLE
STL_KO	Source of defensive programming error: STL internal data corrupted	No state change

Additional information: there is no specific CPU initialization function for CPU test modules.

Note: This function uses HW CRC as explained in [Section 4.3.4 CRC resources](#).



7.2.2 CPU Arm® core testing APIs

7.2.2.1 STL_Status_t STL_SCH_RunCpuTMx

Description: runs CPU test module x. Used only for single test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_RunCpuTMx(STL_TmStatus_t *pSingleTmStatus) where TMx can be one of TM1, TM1L, TM2, TM3, TM4, TM5, TM6, TM7, TM7L, TM8, TM9, TM10 or TM11.

Table 8. STL_SCH_RunCpuTMx input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
CPU_TMx_CONFIGURED	*pSingleTmStatus	See Caution

Table 9. STL_SCH_RunCpuTMx output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_PASSED	-	CPU_TMx_CONFIGURED
		STL_FAILED	-	
		STL_ERROR	Source of defensive programming error: - STL internal data corrupted - software is not executed with privileged level for CPU TM7 - software is not executed in thread mode for CPU TM7	
STL_KO	Possible source of defensive programming error: *pSingleTmStatus = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change

Note: The test is executed whatever the CPU TMx enable status (refer to [Section 7.2.5 Multiple testing APIs](#)).



7.2.3 Flash memory testing APIs

7.2.3.1 *STL_Status_t* *STL_SCH_InitFlash*

Description: initializes Flash memory test (see [Section 7.3 State machines](#)).

Declaration: *STL_Status_t* *STL_SCH_InitFlash*(*STL_TmStatus_t* **pSingleTmStatus*)

Table 10. *STL_SCH_InitFlash* input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
FLASH_IDLE FLASH_INIT FLASH_CONFIGURED	* <i>pSingleTmStatus</i>	See Caution

Table 11. *STL_SCH_InitFlash* output information

<i>STL_Status_t</i> return value		<i>*pSingleTmStatus</i> output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	FLASH_INIT
STL_KO	Possible source of defensive programming error: <i>pSingleTmStatus</i> = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change



7.2.3.2 **STL_Status_t STL_SCH_ConfigureFlash**

Description: configures Flash memory test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_ConfigureFlash(STL_TmStatus_t *pSingleTmStatus, STL_MemConfig_t *pFlashConfig)

Table 12. STL_SCH_ConfigureFlash input information

Allowed state(s)	Parameter	
	Value	Comment(s)
FLASH_INIT	*pSingleTmStatus	See Caution
		- This pointer contains the Flash memory configuration - See Caution
		Field
		Comment(s)
		- Pointer to Flash memory subset - A section cannot overlap with the CRC area - See Caution
		Field
		Comment(s)
		- Start subset address in bytes - Cannot be lower than ROM_START and higher than CRC_START address - For further details see Section 4.1.3 Flash memory tests
		Field
		Comment(s)
FLASH_INIT	*pFlashConfig	*pSubset
		- End subset address in bytes - Cannot be lower than ROM_START and higher than CRC_START address - Needs to be higher than StartAddr - For further details see Section 4.1.3 Flash memory tests
		*pNext
		- Pointer to next Flash memory subset - Must be set to NULL for the last subset - See Caution
FLASH_INIT		NumSectionsAtomic
		- Number of Flash memory sections to be tested during an atomic test - Set to 1, as minimum (one section per test) - If the value is higher than the number of sections in all subsets, all Flash memory subsets are tested in one pass


Table 13. STL_SCH_ConfigureFlash output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	FLASH_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: • STL internal data corrupted • State not allowed • Wrong configuration detected	No state change
STL_KO	Possible source of defensive programming error: • <i>pSingleTmStatus</i> = NULL • <i>pFlashConfig</i> = NULL • STL internal data corrupted	Not relevant	Value must not be used	No state change

Additional Information: in the case of a return value set to STL_KO or *pSingleTmStatus set to STL_ERROR, the Flash memory configuration is not applied.

7.2.3.3

STL_Status_t STL_SCH_RunFlashTM

Description: runs Flash memory test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_RunFlashTM(STL_TmStatus_t *pSingleTmStatus)

Table 14. STL_SCH_RunFlashTM input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
FLASH_CONFIGURED	*pSingleTmStatus	See Caution

Table 15. STL_SCH_RunFlash™ output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_PASSED	-	FLASH_CONFIGURED
		STL_PARTIAL_PASSED	-	FLASH_CONFIGURED
		STL_FAILED	-	FLASH_CONFIGURED
		STL_NOT_TESTED	All subsets are already tested	FLASH_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
STL_KO	Possible source of defensive programming error: <i>pSingleTmStatus</i> = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change

7.2.3.4

STL_Status_t STL_SCH_ResetFlash

Description: resets Flash memory test (see Section 7.3 State machines).

Declaration: STL_Status_t STL_SCH_ResetFlash(STL_TmStatus_t *pSingleTmStatus)

Table 16. STL_SCH_ResetFlash input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
FLASH_CONFIGURED	*pSingleTmStatus	See Caution

Table 17. STL_SCH_ResetFlash output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	Configuration successfully applied	FLASH_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
STL_KO	Possible source of defensive programming error: <i>pSingleTmStatus</i> = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change



Additional information

- Once all subsets are tested, the user needs to reset the test module to perform the Flash memory test again.
- In the case of a return value set to STL_KO or **pSingleTmStatus* set to STL_ERROR, the Flash reset is not applied.

7.2.3.5

STL_Status_t STL_SCH_DeInitFlash

Description: deinitializes Flash memory test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_DeInitFlash(STL_TmStatus_t *pSingleTmStatus)

Table 18. STL_SCH_DeInitFlash input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
FLASH_IDLE FLASH_INIT FLASH_CONFIGURED	<i>*pSingleTmStatus</i>	See Caution

Table 19. STL_SCH_DeInitFlash output information

STL_Status_t return value		<i>*pSingleTmStatus</i> output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	FLASH_IDLE
STL_KO	Possible source of defensive programming error: • <i>pSingleTmStatus</i> = NULL • STL internal data corrupted	Not relevant	Value must not be used	No state change



7.2.4 RAM testing APIs

7.2.4.1 *STL_Status_t* *STL_SCH_InitRam*

Description: initializes RAM test (see [Section 7.3 State machines](#)).

Declaration: *STL_Status_t* *STL_SCH_InitRam*(*STL_TmStatus_t* *pSingleTmStatus)

Table 20. *STL_SCH_InitRam* input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
RAM_IDLE RAM_INIT RAM_CONFIGURED	*pSingleTmStatus	See Caution

Table 21. *STL_SCH_InitRam* output information

<i>STL_Status_t</i> return value		<i>*pSingleTmStatus</i> output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	RAM_INIT
STL_KO	Possible source of defensive programming error: • <i>pSingleTmStatus</i> = NULL • STL internal data corrupted	Not relevant	Value must not be used	No state change



7.2.4.2 **STL_Status_t STL_SCH_ConfigureRam**

Description: configures RAM test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_ConfigureRam(STL_TmStatus_t *pSingleTmStatus, STL_MemConfig_t *pRamConfig)

Table 22. STL_SCH_ConfigureRam input information

Allowed state(s)	Parameter	
	Value	Comment(s)
RAM_INIT	*pSingleTmStatus	See Caution
		This pointer contains the RAM configuration.
		See Caution
		Field
		Comment(s)
		- Pointer to RAM subset - A subset cannot overlap with the RAM backup buffer if defined - See Caution
		Field
		Comment(s)
		- Start subset address in bytes - Start address must be 32-bit aligned - RAM subset must be inside RAM area - Cannot be lower than RAM_START and higher than RAM_END address
		Field
		Comment(s)
		- End subset address in bytes - Higher than StartAddr - Cannot be lower than RAM_START and higher than RAM_END address - Subset size (EndAddr – StartAddr) needs to be multiple of 2 * RAM_BLOCK_SIZE, 32 bytes - Subset cannot overlap with the RAM backup buffer, if defined
		Field
		Comment(s)
		- Pointer to next RAM subset - Must be set to NULL for the last subset - See Caution
		Field
		Comment(s)
		- Number of RAM sections to be tested during an atomic test - Set to 1, as minimum (one section per test) - If the value is higher than the number of sections in all subsets, all RAM subsets are tested in one pass

Table 23. STL_SCH_ConfigureRam output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	RAM_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: • STL internal data corrupted • State not allowed • Wrong configuration detected	No state change
STL_KO	Possible source of defensive programming error: • <i>pSingleTmStatus</i> = NULL • <i>pRamConfig</i> = NULL • STL internal data corrupted	Not relevant	Value must not be used	No state change

Additional Information: in case of return value set to STL_KO or *pSingleTmStatus set to STL_ERROR, the RAM configuration is not applied.


7.2.4.3
STL_Status_t STL_SCH_RunRamTM

Description: runs RAM test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_RunRamTM(STL_TmStatus_t *pSingleTmStatus)

Table 24. STL_SCH_RunRamTM input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
RAM_CONFIGURED	*pSingleTmStatus	See Caution

Table 25. STL_SCH_RunRamTM output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_PASSED	-	RAM_CONFIGURED
		STL_PARTIAL_PASSED	-	RAM_CONFIGURED
		STL_FAILED	-	RAM_CONFIGURED
		STL_NOT_TESTED	All subsets are already tested	RAM_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
STL_KO	Possible source of defensive programming error: *pSingleTmStatus = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change


7.2.4.4
STL_Status_t STL_SCH_ResetRam

Description: resets RAM test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_ResetRam(STL_TmStatus_t *pSingleTmStatus)

Table 26. STL_SCH_ResetRam input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
RAM_CONFIGURED	*pSingleTmStatus	See Caution

Table 27. STL_SCH_ResetRam output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	Configuration successfully applied	RAM_CONFIGURED
		STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
STL_KO	Possible source of defensive programming error: *pSingleTmStatus = NULL - STL internal data corrupted	Not relevant	Value must not be used	No state change

Additional information

- Once all subsets are tested, the user needs to reset the test module to perform the RAM test again.
- In the case of a return value set to STL_KO or *pSingleTmStatus set to STL_ERROR, the RAM reset is not applied.


7.2.4.5
STL_Status_t STL_SCH_DeInitRam

Description: deinitializes RAM test (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_DeInitRam(STL_TmStatus_t *pSingleTmStatus)

Table 28. STL_SCH_DeInitRam input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
RAM_IDLE RAM_INIT RAM_CONFIGURED	*pSingleTmStatus	See Caution

Table 29. STL_SCH_DeInitRam output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Value	Comment(s)	
STL_OK	Function successfully executed	STL_NOT_TESTED	-	RAM_IDLE
STL_KO	Possible source of defensive programming error: • <i>pSingleTmStatus</i> = NULL • STL internal data corrupted	Not relevant	Value must not be used	No state change



7.2.5 Multiple testing APIs

7.2.5.1 **STL_Status_t STL_SCH_InitAllTM**

Description: initializes Flash memory and RAM test modules, and enables/disables each test module (see Section 7.3 State machines).

Declaration: STL_Status_t STL_SCH_InitAllTM(STL_TmListStatus_t *pTmListStatus, STL_TmListEnable_t *pTmListEnable)

Table 30. STL_SCH_InitAllTM input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
CPU TMx: • CPU_TMx_CONFIGURED Flash TM: • FLASH_IDLE • FLASH_INIT • FLASH_CONFIGURED RAM TM: • RAM_IDLE • RAM_INIT • RAM_CONFIGURED	*pTmListStatus	See Caution
	pTmListEnable	This pointer gives information on if the application enables or disables each internal STL test module (CPU TMx, Flash TM, RAM TM). The user test(s) is (are) directly configured and so enabled inside STL_SCH_ConfigureAllTM().

Table 31. STL_SCH_InitAllTM output information

STL_Status_t return value		*pSingleTmStatus output		Returned state
Value	Comment(s)	Field and Value	Comment(s)	
STL_OK	Function successfully executed	aCpuTmStatus[STL_CPU_TMx_IDX] = STL_NOT_TESTED	-	CPU TMx: • CPU_TMx_CONFIGURED
		FlashTmStatus = STL_NOT_TESTED	-	Flash TM: • If test enabled: FLASH_INIT • If test not enabled: FLASH_IDLE
		RamTmStatus = STL_NOT_TESTED	-	RAM TM: • If test enabled: RAM_INIT • If test not enabled: RAM_IDLE
STL_KO	Possible source of defensive programming error: • pTmListStatus = NULL • pTmListEnable = NULL • If user test defined, number of user tests result (STL_UserTmStatus_t) is different from the number of user tests configuration (STL_UserTmConfig_t) • STL internal data corrupted	Not relevant	Value must not be used	No state change


7.2.5.2
STL_Status_t STL_SCH_ConfigureAllTM

Description: configures Flash memory and RAM test modules, and registers User tests (see [Section 7.3 State machines](#)).

Declaration: STL_Status_t STL_SCH_ConfigureAllTM(STL_TmListStatus_t *pTmListStatus, STL_MemConfig_t *pFlashConfig, STL_MemConfig_t *pRamConfig, const STL_UserTmConfig_t *pUserTmConfig)

Table 32. STL_SCH_ConfigureAllTM input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
Flash TM: • FLASH_INIT RAM TM: • RAM_INIT	*pTmListStatus	See Caution
	*pFlashConfig	- Same information as for MemConfig_t *pFlashConfig described in STL_SCH_ConfigureFlash() - Pointer could be set to NULL if TM Flash is not enabled
	*pRamConfig	- Same information as for MemConfig_t *pRamConfig described in STL_SCH_ConfigureRam() - Pointer could be set to NULL if TM RAM is not enabled
	*pUserTmConfig	- Application must set a valid pointer, if user tests are defined - In the case of user tests, the structure has to define the same number of user test result (STL_UserTmStatus_t) as the number of user tests configuration (STL_UserTmConfig_t)


Table 33. STL_SCH_ConfigureAllTM output information

STL_Status_t return value		*pTmListStatus output			Returned state
Value	Comment(s)	Field	Value	Comment(s)	
STL_OK	Function successfully executed	.aCpuTmStatus[STL_CPU_TMx_ID X]	Not relevant	Value must not be used	No state change
			STL_NOT_TESTED	Configuration successfully applied	FLASH_CONFIGURED
				Test is not enabled	No state change
		.FlashTmStatus	STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Wrong configuration detected	No state change
			STL_NOT_TESTED	Configuration successfully applied	RAM_CONFIGURED
				Test is not enabled	No state change
.RamTmStatus	STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Wrong configuration detected	No state change		
.pUserTmStatus	STL_NOT_TESTED	-	-		
STL_KO	Possible source of defensive programming error: - pTmListStatus = NULL - Flash TM: If pFlashConfig = NULL and test enabled - RAM TM: If pRamConfig = NULL and test enabled - If user test defined, number of user tests result (STL_UserTmStatus_t) is different from the number of user tests configuration (STL_UserTmConfig_t)	.aCpuTmStatus[STL_CPU_TMx_ID X]	Not relevant	Value must not be used	CPU TMx: No state change
		.FlashTmStatus			Flash TM: No state change
		.RamTmStatus			RAM TM: No state change
		.pUserTmStatus			User TM: not managed at STL level



STL_Status_t return value		*pTmListStatus output			Returned state
Value	Comment(s)	Field	Value	Comment(s)	
STL_KO	• STL internal data corrupted		Not relevant	Value must not be used	

Note: This API has no effect on CPU TMx.

ST Restricted


7.2.5.3
STL_Status_t STL_SCH_RunAllTM

Description: runs all test modules (see Section 7.3 State machines).

Declaration: STL_Status_t STL_SCH_RunAllTM(STL_TmListStatus_t *pTmListStatus)

Table 34. STL_SCH_RunAllTM input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
CPU TMx: CPU_TMx_CONFIGURED Flash TM: - If enabled: FLASH_CONFIGURED - If not enabled: FLASH_IDLE, FLASH_INIT and FLASH_CONFIGURED RAM TM ⁽¹⁾ : - if enabled: RAM_CONFIGURED - If not enabled: RAM_IDLE, RAM_INIT and RAM_CONFIGURED	*pTmListStatus	See Caution

1. See STL_SCH_InitAllTM in Section 7.2.5 Multiple testing APIs

Table 35. STL_SCH_RunAllTM output information

STL_Status_t return value		*pTmListStatus output			Returned state
Value	Comment(s)	Field	Value	Comment(s)	
STL_OK	Function successfully executed	.aCpuTmStatus[STL_CPU_TMx_ID X]	STL_PASSED	-	CPU TMx: CPU_TMx_CONFIGURED
			STL_FAILED	-	
			STL_ERROR	Source of defensive programming error: - STL internal data corrupted - Software is not executed with privileged level for CPU TM7, Software is not executed in thread mode for CPU TM7	
			STL_NOT_TESTED	Only if CPU TMx is not enabled	CPU TMx: No state change
		.FlashTmStatus	STL_PASSED	-	FLASH_CONFIGURED
			STL_PARTIAL_PASSED	-	
			STL_FAILED	-	No state change
			STL_NOT_TESTED	Test not enabled	
				All subsets are already tested	FLASH_CONFIGURED



STL_Status_t return value		*pTmListStatus output			Returned state
Value	Comment(s)	Field	Value	Comment(s)	
STL_OK	Function successfully executed	.FlashTmStatus	STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
			STL_PASSED	-	RAM_CONFIGURED
			STL_PARTIAL_PASSED	-	
		STL_FAILED	-	No state change	
		STL_NOT_TESTED	Test not enabled	RAM_CONFIGURED	
			All subsets are already tested		
		.RamTmStatus	STL_ERROR	Possible source of defensive programming error: - STL internal data corrupted - State not allowed - Configuration corrupted	No state change
.pUserTmStatus	Unknown		Depends on user test	-	
STL_KO	Possible source of defensive programming error: - pTmListStatus = NULL - If user test defined, number of user tests result (STL_UserTmStatus_t) is different from the number of user tests configuration (STL_UserTmConfig_t) - STL internal data corrupted	.aCpuTmStatus[STL_CPU_TMx_ID X]	Not relevant	Value must not be used	CPU TMx: no state change
		.FlashTmStatus			Flash TM: no state change
		.RamTmStatus			RAM TM: no state change
		.pUserTmStatus			User TM: not managed at STL level



7.2.6 Artificial-failing APIs

7.2.6.1 *STL_Status_t STL_SCH_StartArtifFailing*

Description: sets artificial-failing configuration and starts artificial-failing feature.

Declaration: `STL_Status_t STL_SCH_StartArtifFailing(const STL_ArtifFailingConfig_t *pArtifFailingConfig)`

Behavior: The next functions calls are executed normally. If the *STL_Status_t* return value is set to *STL_OK*, the test module status (**pSingleTmStatus*, **pTmListStatus*) is forced to a configured value. It is forced for each STL test module, even if not enabled. It applies to each STL internal test module (not user ones).

Table 36. *STL_SCH_StartArtifFailing* input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
CPU TMx: • CPU_TMx_CONFIGURED Flash TM: • FLASH_IDLE • FLASH_INIT • FLASH_CONFIGURED RAM TM • RAM_IDLE • RAM_INIT • RAM_CONFIGURED	<i>*pArtifFailingConfig</i>	-

Table 37. *STL_SCH_StartArtifFailing* output information

STL_Status_t return value	Comment(s)	Output	Comment(s)
STL_OK	Function successfully executed	No output parameter	No state change
STL_KO	Possible source of defensive programming error: • <i>pArtifFailingConfig</i> = NULL • configured values are not set for each test module • STL internal data corrupted		


7.2.6.2
STL_Status_t STL_SCH_StopArtifFailing

Description: stops the artificial-failing feature.

Declaration: STL_Status_t STL_SCH_StopArtifFailing(void)

Table 38. STL_SCH_StopArtifFailing input information

Allowed state(s)	Parameter(s)	
	Value	Comment(s)
CPU TMx: - CPU_TMx_CONFIGURED Flash TM: - FLASH_IDLE - FLASH_INIT - FLASH_CONFIGURED RAM TM - RAM_IDLE - RAM_INIT - RAM_CONFIGURED	-	-

Table 39. STL_SCH_StopArtifFailing output information

STL_Status_t return value	Comment(s)	Output	Comment(s)
STL_OK	Function successfully executed	No output parameter	No state change
STL_KO	Possible source of defensive programming error: STL internal data corrupted		

7.3 State machines

Each CPU test module has its own state machine diagram linked to the CPU test APIs.

Figure 12. State machine diagram - CPU test APIs

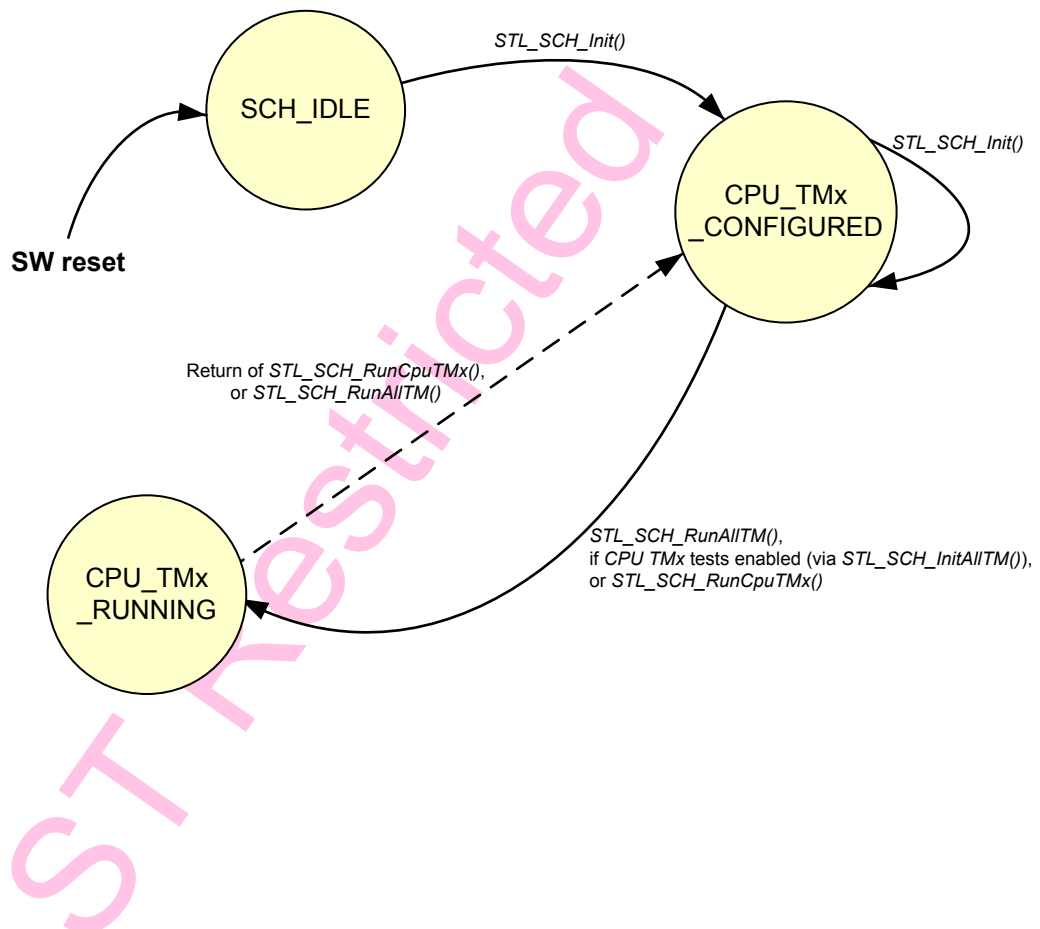
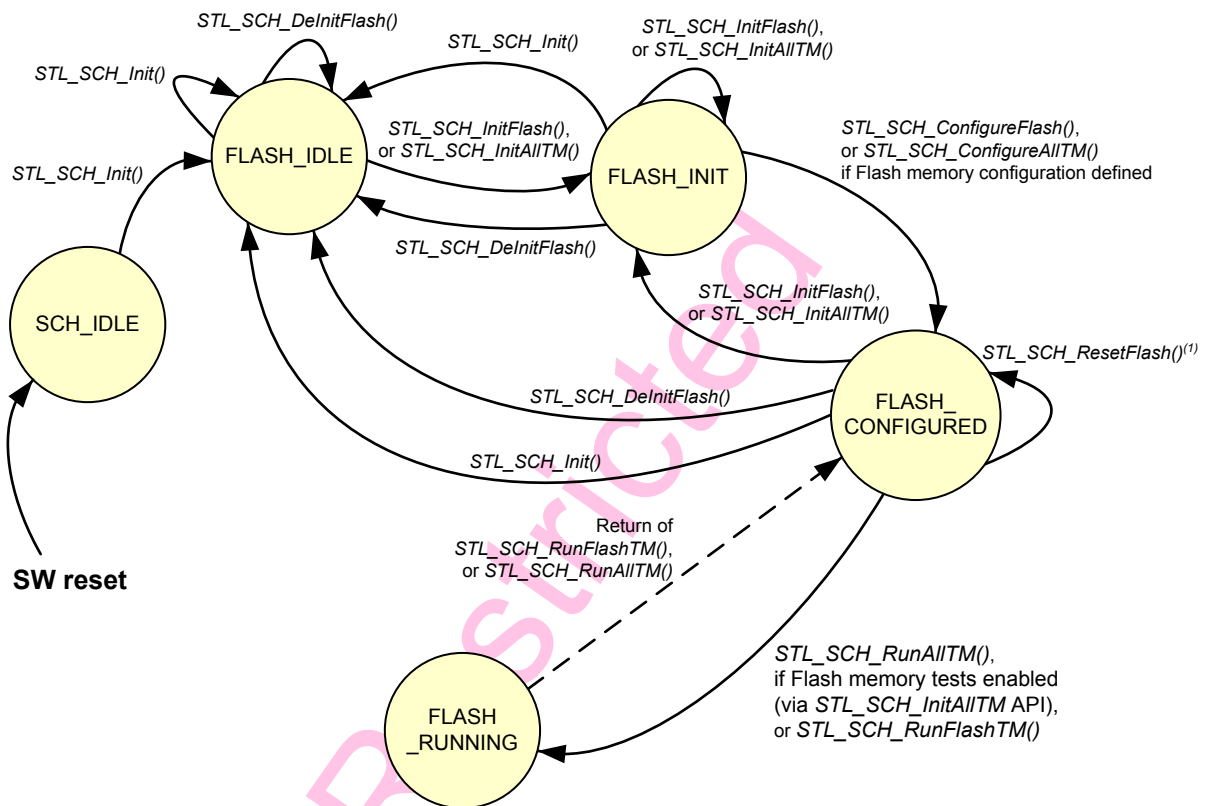
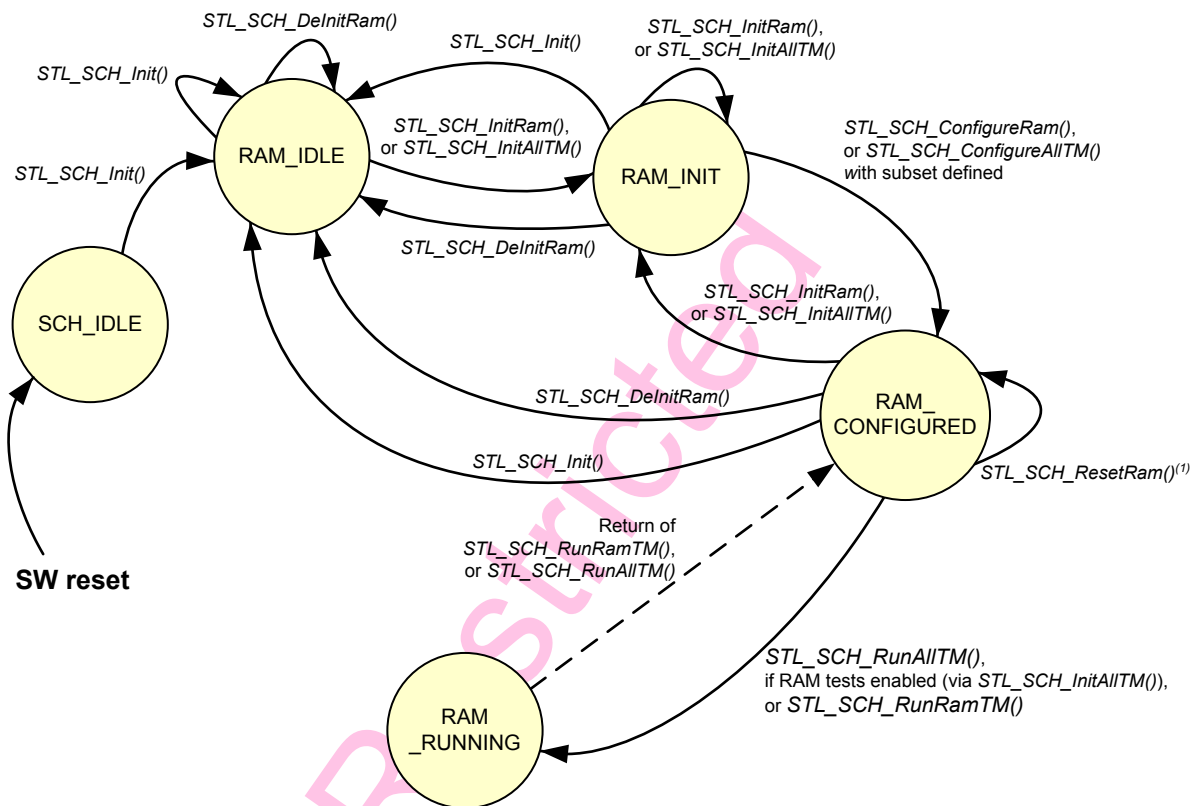


Figure 13. State machine diagram - Flash memory test APIs


1. Once all subsets are tested, the user needs to reset the Flash test module to perform the test again.

Figure 14. State machine diagram - RAM test APIs


1. Once all subsets are tested, the user needs to reset the RAM test module to perform the test again.



7.4 API usage

The user application must:

- Maintain the availability and integrity of pointers passed as parameters during the tests (the STL does not copy the pointer content, and directly accesses memory addresses defined by the application)
- Check the status of function return (*STL_Status_t*), before checking the test result (*STL_TmStatus_t* or *STL_TmListStatus_t*). See the example in the delivered applications.

7.5 User parameters

In addition to parameters set directly inside the APIs, there are few parameters to be customized in the file *stl_user_param_template.c*. They can be found in the code, with the following comments:

```
/* customisable */
```

Extract from *stl_user_param_template.c*:

```
/* Flash configuration */
#define STL_ROM_START (0x08000000U) /* customizable */
#define STL_ROM_END   (0x0803FFFFU) /* customizable */
```

The customization depends upon the STM32 product and the user choice.

```
/* TM RAM Backup Buffer configuration */
...
/* User shall locate the buffer in RAM */
/* The RAM backup buffer is placed in "backup_buffer_section". */
/* "backup_buffer_section" section is defined in scatter file */
```

Customization depends upon the user choice.

The remaining user parameters are defined by flags, and can be checked inside the files:

- *stl_user_param_template.c*: use of RAM backup buffer or not
- *stl_util.c*: use of SW or HW CRC computation
- *stl_stm32_hw_config.h*: if CRC HW is used, choose the right CRC IP configuration according to the STM32 device

See [Section 5.5.2 Steps to build an application from scratch](#) to check flag configuration.



7.6 Test examples

Figure 15. Single testing example

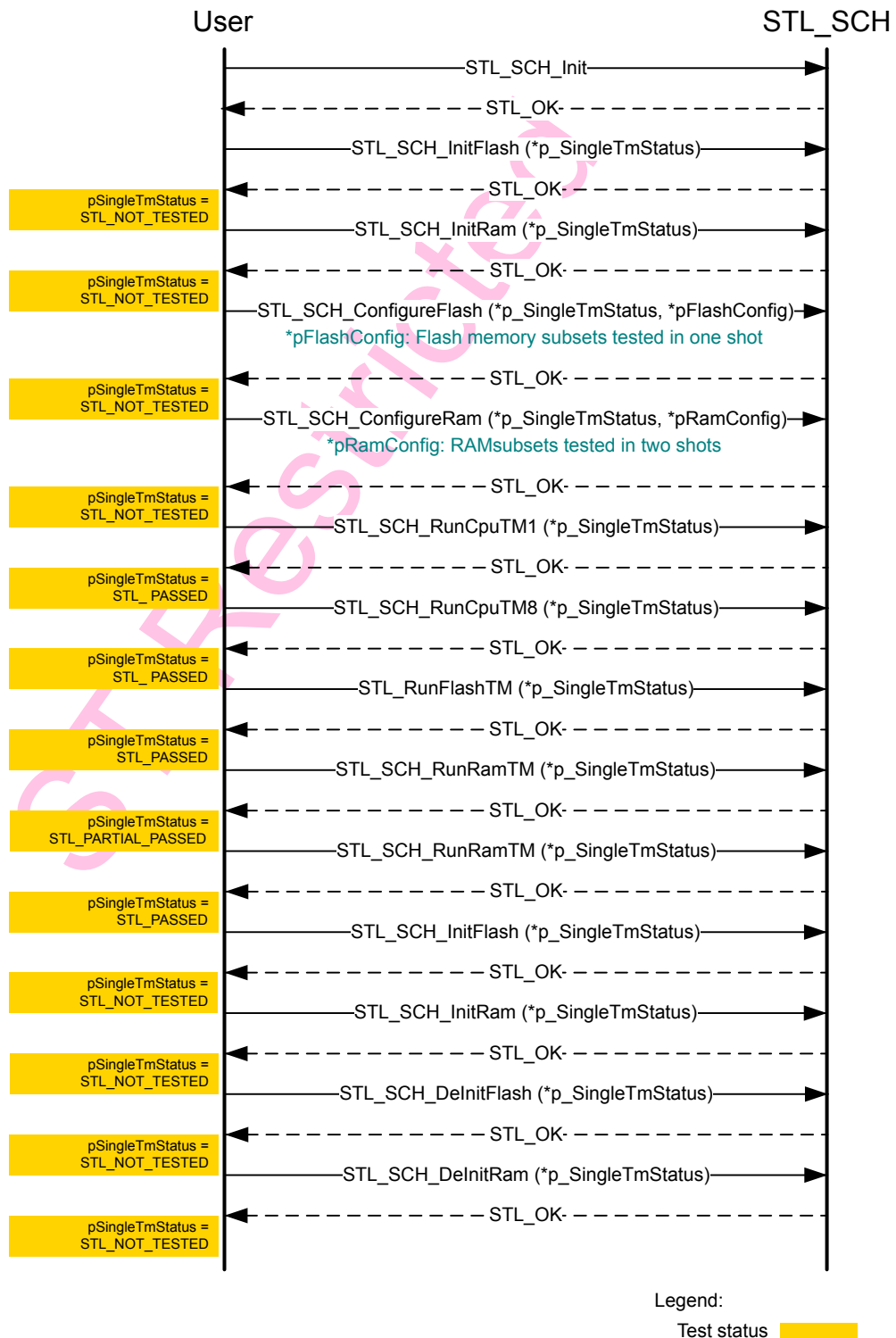


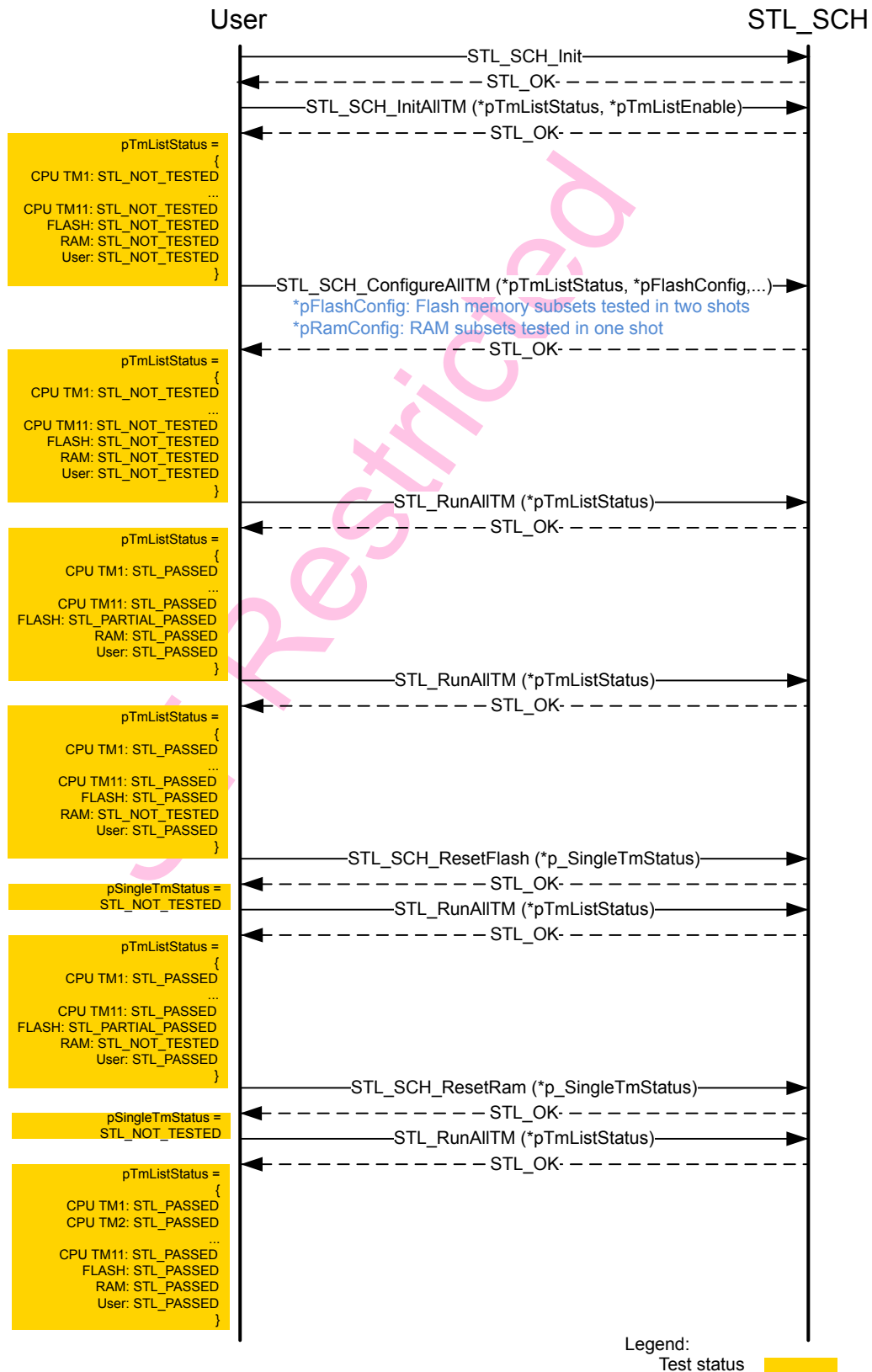
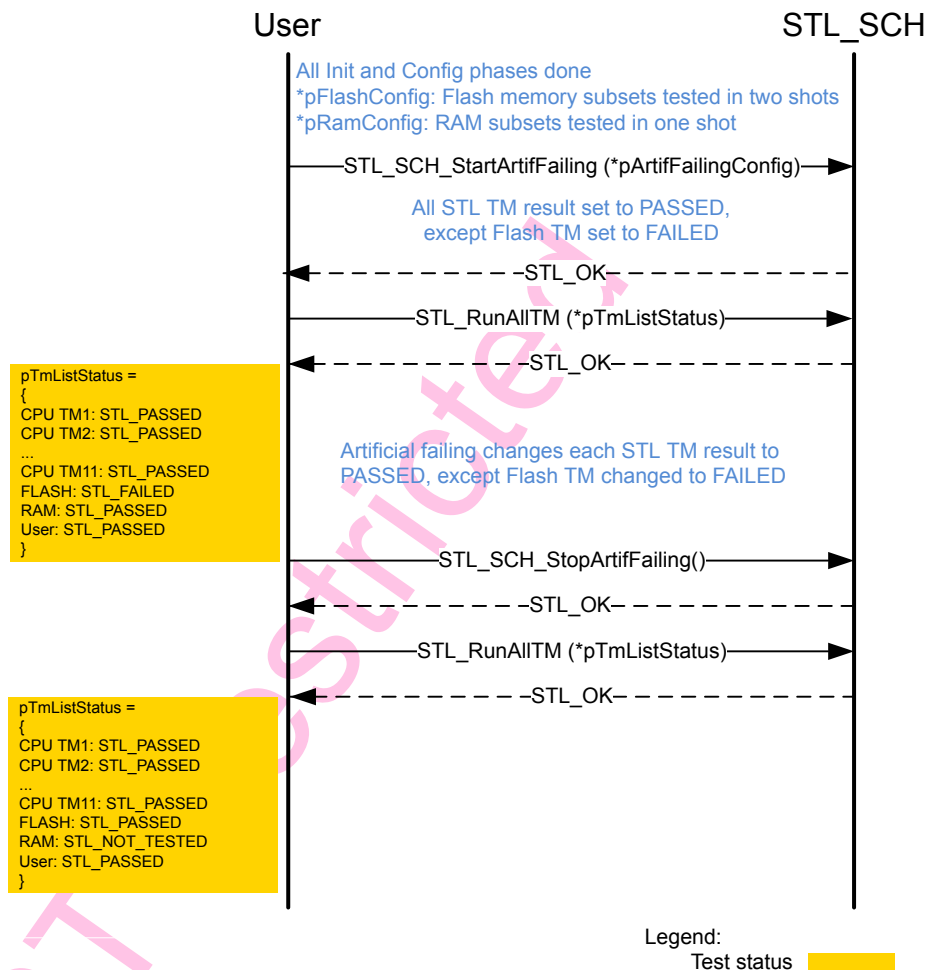

Figure 16. Multiple testing example


Figure 17. Artificial failing testing example




7.7 Details of testing examples

7.7.1 Flash memory single test example

Figure 18. Flash memory single test example shows a Flash memory test:

- Use of two Flash memory subsets
- Use of functions
 - STL_SCH_RunFlashTM → only the Flash memory test module is executed
 - STL_SCH_ResetFlash
- Function return value
- Flash memory test module result value: pSingleTmStatus → in this case, it contains the result of the Flash memory test

7.7.2 Flash memory multiple test example

Figure 19. Flash memory multiple test example shows a Flash memory test inside multiple test:

- All test modules are enabled
- Two Flash memory subsets are used
- Use of functions
 - STL_SCH_RunAllTM → all enabled test modules are executed
 - STL_SCH_ResetFlash
- Function return value
- Test module result value: pTmListStatus, which contains test result value for each test module

7.7.3 RAM single test example

Figure 20. RAM single test example shows a RAM single test:

- Use of two RAM subsets
- Use of functions:
 - STL_SCH_RunRAMTM → only the RAM test module is executed
 - STL_SCH_ResetRam
- Function return value
- RAM Test Module result value: pSingleTmStatus → in this case, it contains the result of the RAM memory test

7.7.4 RAM multiple test example

Figure 21. RAM multiple test example shows a RAM test inside a multiple test:

- All test modules are enabled
- Use of two RAM subsets
- Use of functions:
 - STL_SCH_RunAllTM → all enabled test modules are executed
 - STL_SCH_ResetRam
- Function return value
- Test module results value: pTmListStatus, which contains test result values for each test module



Figure 18. Flash memory single test example

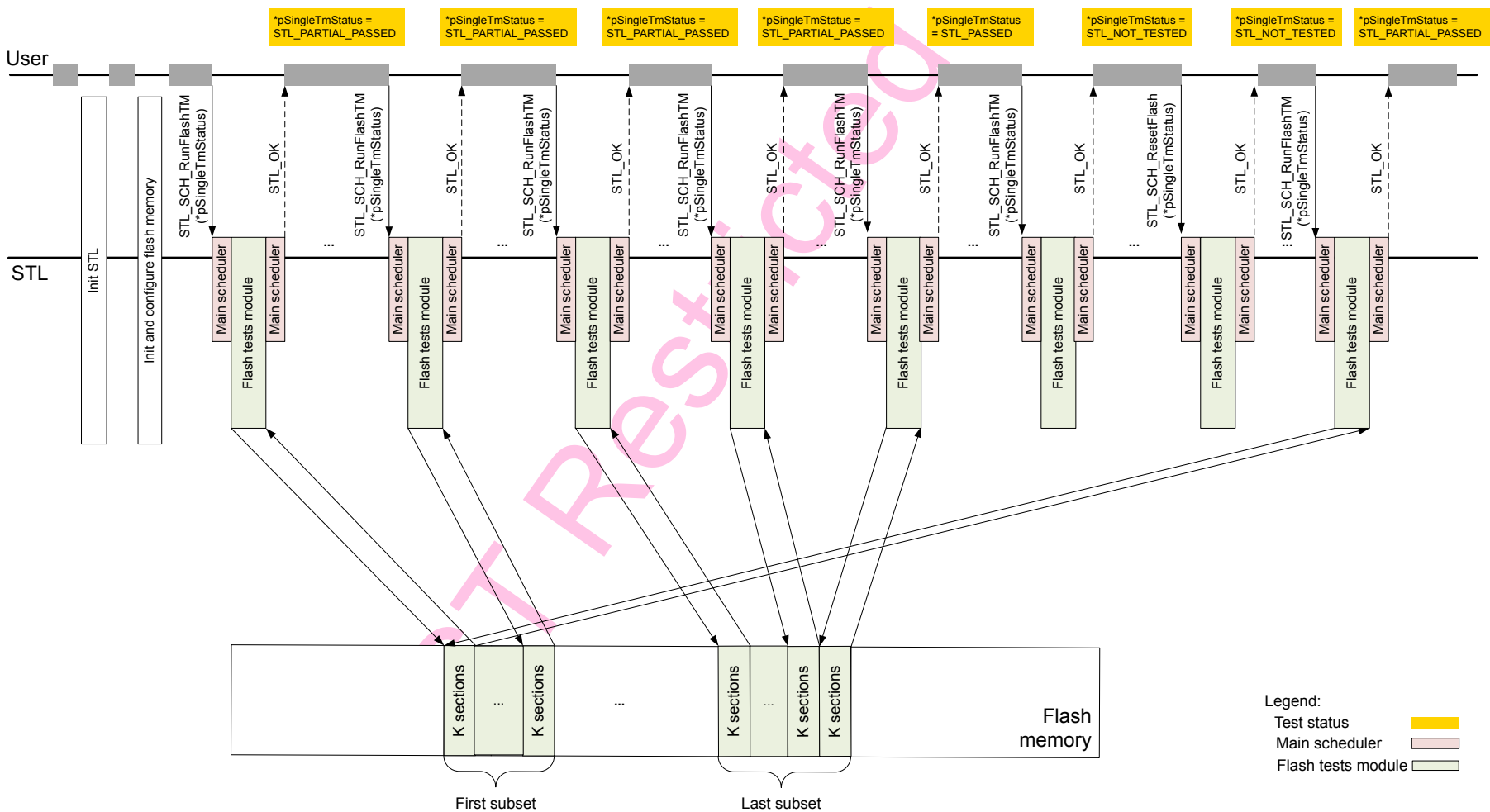




Figure 19. Flash memory multiple test example

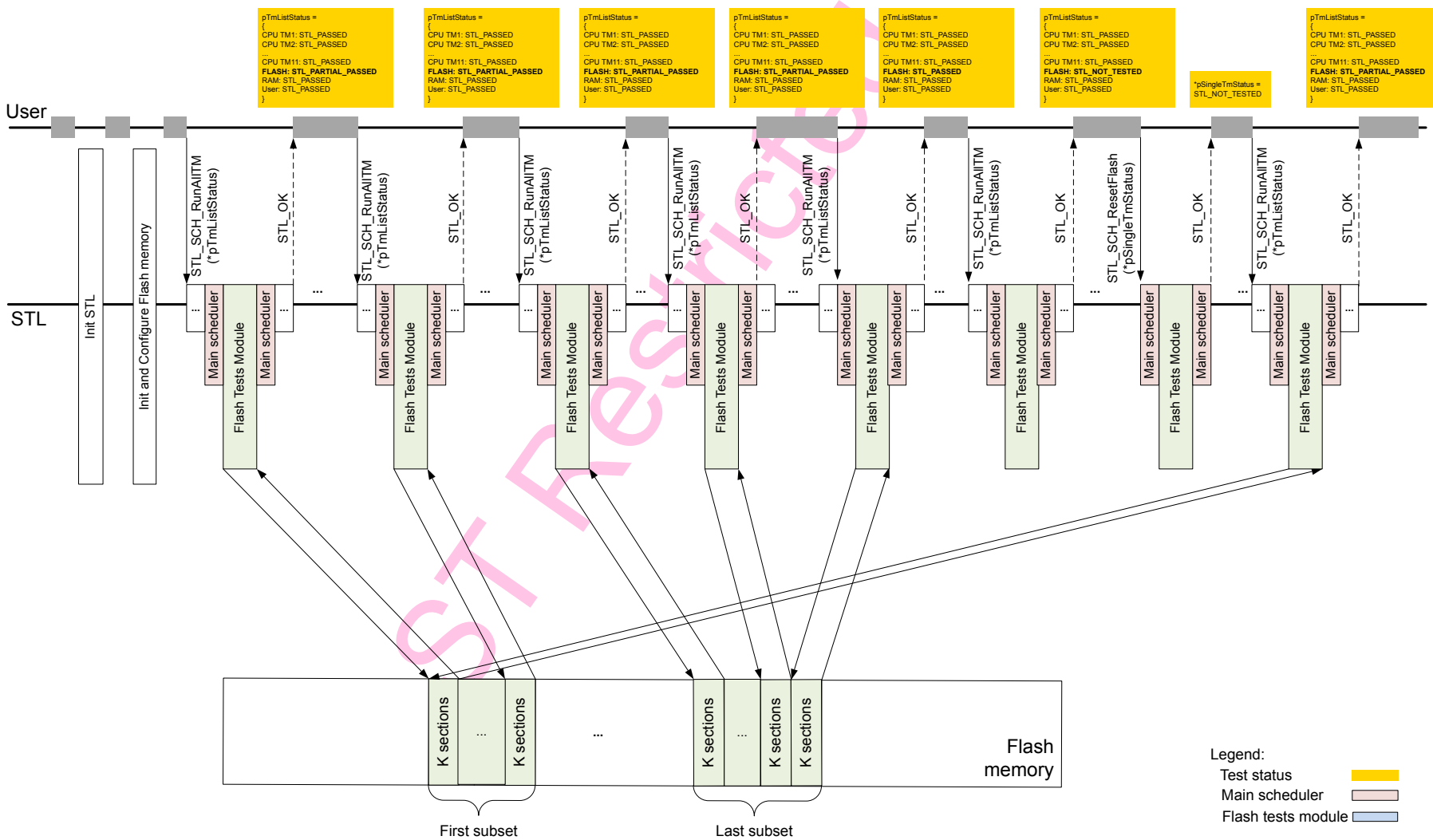
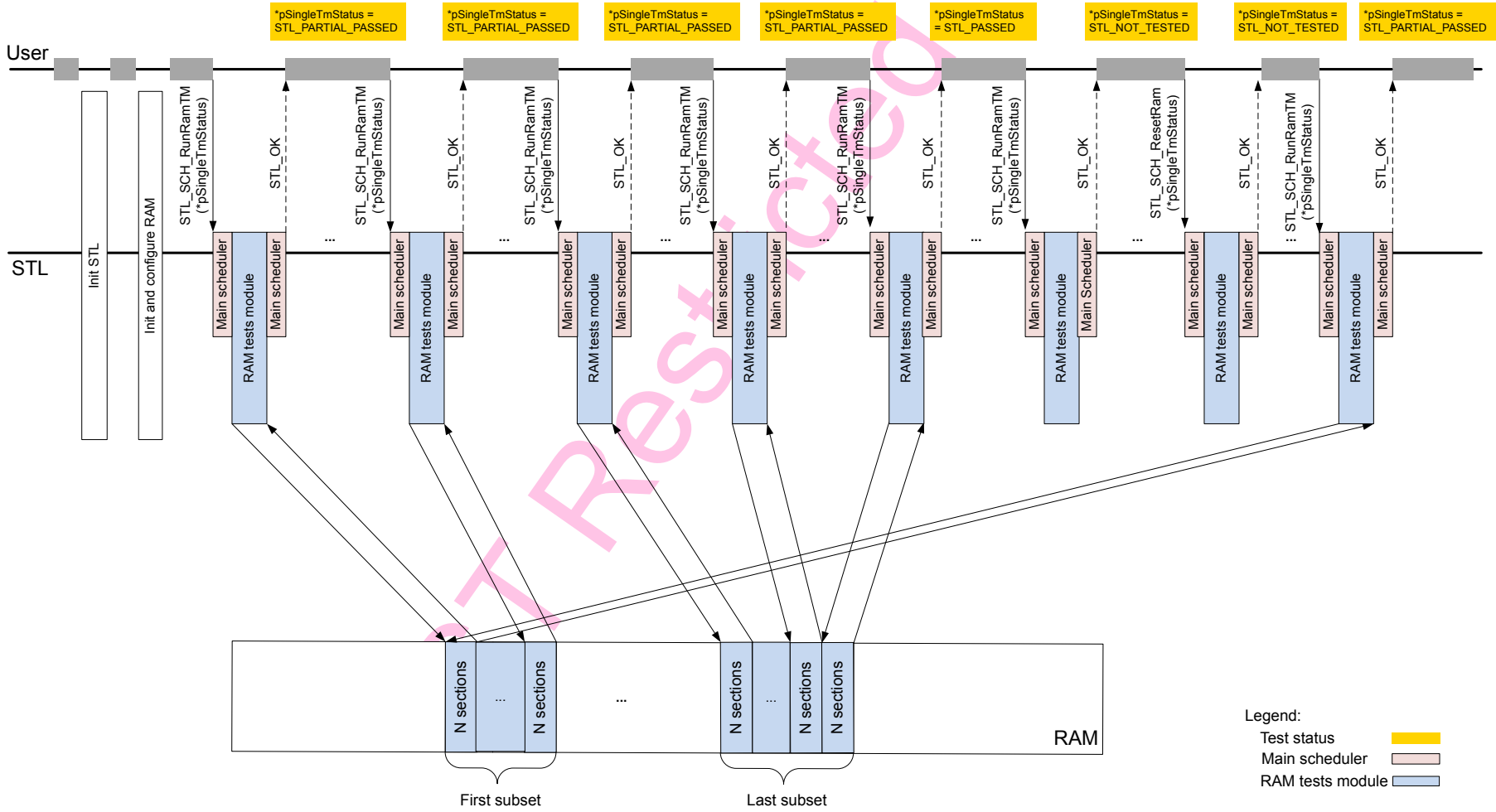


Figure 20. RAM single test example



ST Restricted - Subject to Non-Disclosure Agreement – Do not copy





8 STL: execution timing details

Refer also to the information in [Section 4.2.1 STL execution timing summary](#).

The data in [Table 40](#) is obtained with the following test set-up:

- STL library compilation details, detailed in [Section 5.5 Application: compilation process](#).
- Projects for performance tests compiled with IAR™ Embedded Workbench for Arm® (EWARM) toolchain v9.20.1
- Compiled SW configuration with:
 - AHB clock set to 84 MHz
 - Flash latency set to 2 wait states
 - Flash prefetch enabled
 - Nucleo STM32F429ZI, board MB1137 Rev B

Table 40. Integration tests

Test	Duration		Tested memory
	HW CRC	SW CRC ⁽¹⁾	
STL_SCH_InitAllTM()	18 µs	18 µs	-
STL_SCH_ConfigureAllTM()	20 µs	20 µs	-
STL_SCH_RunAllTM() ²	138788 µs	141390 µs	262111 bytes RAM 17408 bytes Flash memory
STL_SCH_InitFlash()	9 µs	9 µs	-
STL_SCH_ConfigureFlash()	11 µs	11 µs	-
STL_SCH_RunFlashTM()	588 µs	3186 µs	17408 bytes Flash memory
STL_SCH_InitRam()	9 µs	9 µs	-
STL_SCH_ConfigureRam()	12 µs	13 µs	-
STL_SCH_RunRamTM() ³	137489 µs	137489 µs	262111 bytes RAM
STL_SCH_RunCpuTM1()	309 µs	311 µs	-
STL_SCH_RunCpuTM1L()	46 µs	47 µs	-
STL_SCH_RunCpuTM2()	52 µs	52 µs	-
STL_SCH_RunCpuTM3()	50 µs	49 µs	-
STL_SCH_RunCpuTM4()	33 µs	33 µs	-
STL_SCH_RunCpuTM5()	33 µs	33 µs	-
STL_SCH_RunCpuTM6()	28 µs	28 µs	-
STL_SCH_RunCpuTM7()	23 µs	23 µs	-
STL_SCH_RunCpuTM7L()	14 µs	14 µs	-
STL_SCH_RunCpuTM8() ⁴	24 µs	25 µs	-
STL_SCH_RunCpuTM9()	69 µs	69 µs	-
STL_SCH_RunCpuTM10()	67 µs	66 µs	-
STL_SCH_RunCpuTM11()	77 µs	77 µs	-

1. Test duration increased because of CRC algorithm (see [Section 4.1.1 Scheduler principle](#)).

2. Default configuration: STL_DISABLE_RAM_BCKUP_BUF not enabled and STL_ENABLE_IT not enabled.

3. Default configuration: STL_ENABLE_IT not enabled.



Revision history

Table 41. Document revision history

Date	Revision	Changes
04-Dec-2018	1	Initial release.
26-Feb-2019	2	Added Section 4.2.4 STL heap usage Updated: • Section 4.3.4 CRC resources • Section 7.2.1 Common APIs (used whatever the test mode) – Table 37. STL_SCH_StartArtifFailing output information – Table 39. STL_SCH_StopArtifFailing output information
12-Sep-2019	3	Updated functional safety documentation framework.
02-Dec-2022	4	Updated: • Section 1 General information • Section 2 STM32Cube overview • Section 3.1 Architecture overview • Section 4.1 STL functional description (first paragraph) • Scheduler and interrupts • Section 4.1.1 Scheduler principle • Section 4.1.2 CPU Arm core tests (TM7 and TM7L availability, Caution, TM8 interrupt now in TM7) • Section 4.1.3 Flash memory tests • Section 4.1.4 RAM tests • Section 4.2.5 STL interrupt masking time • Section 4.2 STL performance data • Section 4.2.1 STL execution timing summary • Section 4.2.2 STL code and data size • Section 4.4.2 Test 2: correct STL error-message processing • Section 5.2.3 STL • Section 5.2.4 User application • Figure 8. Project file structure • Section 5.4.1 Compliance • Section 5.5.1 Steps to build a delivered STL example • Section 5.5.2 Steps to build an application from scratch • Section 6.2.1 Development tool-chains and compilers • Section 7.1 User structures • Section 7.2 User APIs Reformatted all APIs with Description and Declaration presentation, and STL internal data corrupted source of defensive programming error. • Section 7.2.2 CPU Arm® core testing APIs Added: • Flash memory test and interrupts • Section 4.3.1 Function call convention • Section 4.3.6 GE bits of APSR • Section 4.3.8.1 How the STL masks the interrupts • Section 4.3.10 Supported memories • Section 4.3.13 Processor mode • Section 4.3.14 Integration tips when switching to STL_Lib.a v2.0, and associated source files • Glossary at end of document Removed section 'Interrupt and general purpose register' from Section 4.3.8 Interrupt management.



Contents

1	General information	2
1.1	Purpose and scope	2
1.2	Reference documents	2
2	STM32Cube overview	3
2.1	What is STM32Cube?	3
2.2	How does this software complement STM32Cube?	3
3	STL overview	4
3.1	Architecture overview	4
3.2	Supported products	6
4	STL description	7
4.1	STL functional description	7
4.1.1	Scheduler principle	7
4.1.2	CPU Arm® core tests	9
4.1.3	Flash memory tests	10
4.1.4	RAM tests	13
4.1.5	User tests	14
4.2	STL performance data	15
4.2.1	STL execution timing summary	15
4.2.2	STL code and data size	15
4.2.3	STL stack usage	15
4.2.4	STL heap usage	15
4.2.5	STL interrupt masking time	16
4.3	STL user constraints	17
4.3.1	Function call convention	17
4.3.2	Privileged-level	17
4.3.3	RCC resources	17
4.3.4	CRC resources	17
4.3.5	Bit Q of APSR	17
4.3.6	GE bits of APSR	17
4.3.7	FPU resources (Cortex M4 FPU block)	18
4.3.8	Interrupt management	18
4.3.9	DMA	19
4.3.10	Supported memories	19
4.3.11	RAM backup buffer	19
4.3.12	Memory mapping	19



4.3.13	Processor mode	19
4.3.14	Integration tips when switching to STL_Lib.a v2.0, and associated source files	20
4.4	End user integration tests	21
4.4.1	Test 1: correct STL execution	21
4.4.2	Test 2: correct STL error-message processing	21
5	Package description	22
5.1	General description	22
5.2	Architecture	22
5.2.1	STM32Cube HAL	23
5.2.2	Board support package (BSP)	23
5.2.3	STL	23
5.2.4	User application	23
5.2.5	STL integrity	23
5.3	Folder structure	24
5.4	APIs	25
5.4.1	Compliance	25
5.4.2	Dependency	25
5.4.3	Details	25
5.5	Application: compilation process	26
5.5.1	Steps to build a delivered STL example	26
5.5.2	Steps to build an application from scratch	26
6	Hardware and software environment setup	29
6.1	Hardware setup	29
6.2	Software setup	30
6.2.1	Development tool-chains and compilers	30
6.2.2	CRC tool set-up	30
7	STL: User APIs and state machines	31
7.1	User structures	31
7.2	User APIs	33
7.2.1	Common APIs (used whatever the test mode)	33
7.2.2	CPU Arm® core testing APIs	34
7.2.3	Flash memory testing APIs	35
7.2.4	RAM testing APIs	40
7.2.5	Multiple testing APIs	46
7.2.6	Artificial-failing APIs	52
7.3	State machines	54
7.4	API usage	57



7.5	User parameters	57
7.6	Test examples	58
7.7	Details of testing examples	61
7.7.1	Flash memory single test example	61
7.7.2	Flash memory multiple test example	61
7.7.3	RAM single test example	61
7.7.4	RAM multiple test example	61
8	STL: execution timing details	66
	Revision history	67



List of tables

Table 1.	Applicable product	1
Table 2.	STL return information	7
Table 3.	STL execution timings, clock at 84 MHz	15
Table 4.	STL code size and data size (in Bytes)	15
Table 5.	STL maximum interrupt masking information	16
Table 6.	STL_SCH_Init input information	33
Table 7.	STL_SCH_Init output information	33
Table 8.	STL_SCH_RunCpuTMx input information	34
Table 9.	STL_SCH_RunCpuTMx output information	34
Table 10.	STL_SCH_InitFlash input information	35
Table 11.	STL_SCH_InitFlash output information	35
Table 12.	STL_SCH_ConfigureFlash input information	36
Table 13.	STL_SCH_ConfigureFlash output information	37
Table 14.	STL_SCH_RunFlashTM input information	37
Table 15.	STL_SCH_RunFlashTM output information	38
Table 16.	STL_SCH_ResetFlash input information	38
Table 17.	STL_SCH_ResetFlash output information	38
Table 18.	STL_SCH_DeInitFlash input information	39
Table 19.	STL_SCH_DeInitFlash output information	39
Table 20.	STL_SCH_InitRam input information	40
Table 21.	STL_SCH_InitRam output information	40
Table 22.	STL_SCH_ConfigureRam input information	41
Table 23.	STL_SCH_ConfigureRam output information	42
Table 24.	STL_SCH_RunRamTM input information	43
Table 25.	STL_SCH_RunRamTM output information	43
Table 26.	STL_SCH_ResetRam input information	44
Table 27.	STL_SCH_ResetRam output information	44
Table 28.	STL_SCH_DeInitRam input information	45
Table 29.	STL_SCH_DeInitRam output information	45
Table 30.	STL_SCH_InitAllTM input information	46
Table 31.	STL_SCH_InitAllTM output information	46
Table 32.	STL_SCH_ConfigureAllTM input information	47
Table 33.	STL_SCH_ConfigureAllTM output information	48
Table 34.	STL_SCH_RunAllTM input information	50
Table 35.	STL_SCH_RunAllTM output information	50
Table 36.	STL_SCH_StartArtifFailing input information	52
Table 37.	STL_SCH_StartArtifFailing output information	52
Table 38.	STL_SCH_StopArtifFailing input information	53
Table 39.	STL_SCH_StopArtifFailing output information	53
Table 40.	Integration tests	66
Table 41.	Document revision history	67



List of figures

Figure 1.	STL architecture	5
Figure 2.	Single test mode architecture	8
Figure 3.	Multiple-test mode architecture	8
Figure 4.	Flash memory test: CRC principle	11
Figure 5.	Flash memory test: CRC use cases versus program areas	12
Figure 6.	RAM test: usage	14
Figure 7.	Software architecture overview	22
Figure 8.	Project file structure	24
Figure 9.	IAR post-build actions screenshot	27
Figure 10.	CRC tool command line	28
Figure 11.	STM32 Nucleo board example	29
Figure 12.	State machine diagram - CPU test APIs	54
Figure 13.	State machine diagram - Flash memory test APIs	55
Figure 14.	State machine diagram - RAM test APIs	56
Figure 15.	Single testing example	58
Figure 16.	Multiple testing example	59
Figure 17.	Artificial failing testing example	60
Figure 18.	Flash memory single test example	62
Figure 19.	Flash memory multiple test example	63
Figure 20.	RAM single test example	64
Figure 21.	RAM multiple test example	65



Glossary

ADC analog to digital converter	STL self-test library
AEABI Arm® embedded application binary interface	TM test module
API application programming interface	
APSR CPU status register	
BSP board support package	
Class B middle level of regulations targeting safety for home appliances (UL/CSA/IEC 60730-1/60335-1)	
CMSIS Common Microcontroller Software Interface Standard	
CPU central processing unit	
CRC cyclic redundancy check	
DAC Digital to analog converter	
GPIO general purpose input output	
HAL hardware abstraction level	
ICache instruction cache	
IDE integrated development environment	
LL low layer	
MCU microcontroller unit	
MSP main stack pointer	
OPAMP operational amplifier	
PSP process stack pointer	
PWM pulse width modulation	
RAM random access memory	
SDK software development kit	
SG secure gateway	

**IMPORTANT NOTICE – READ CAREFULLY**

STMicroelectronics International NV and its affiliates (“ST”) reserve the right to make changes corrections, enhancements, modifications, and improvements to ST products and/or to this document any time without notice.

This document is provided solely for the purpose of obtaining general information relating to an ST product. Accordingly, you hereby agree to make use of this document solely for the purpose of obtaining general information relating to the ST product. You further acknowledge and agree that this document may not be used in or in connection with any legal or administrative proceeding in any court, arbitration, agency, commission or other tribunal or in connection with any action, cause of action, litigation, claim, allegation, demand or dispute of any kind. You further acknowledge and agree that this document shall not be construed as an admission, acknowledgement or evidence of any kind, including, without limitation, as to the liability, fault or responsibility whatsoever of ST or any of its affiliates, or as to the accuracy or validity of the information contained herein, or concerning any alleged product issue, failure, or defect. ST does not promise that this document is accurate or error free and specifically disclaims all warranties, express or implied, as to the accuracy of the information contained herein. Accordingly, you agree that in no event will ST or its affiliates be liable to you for any direct, indirect, consequential, exemplary, incidental, punitive, or other damages, including lost profits, arising from or relating to your reliance upon or use of this document.

Disclosure of this document to any non-authorized party must be previously authorized by ST only under the provision of a proper confidentiality contractual arrangement executed between ST and you and must be treat as strictly confidential.

At all times you will comply with the following securities rules:

- Do not copy or reproduce all or part of this document.
- Keep this document locked away.
- Further copies can be provided on a “need to know basis”. Contact your local ST Sales Office or document writer.

Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgment, including, without limitation, the warranty provisions thereunder.

In that respect, note that ST products are not designed for use in some specific applications or environments described in above mentioned terms and conditions.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of purchasers’ products.

Information furnished is believed to be accurate and reliable. However, ST assumes no responsibility for the consequences of use of such information nor for any infringement of patents or other rights of third parties which may result from its use. No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2022 STMicroelectronics – All rights reserved